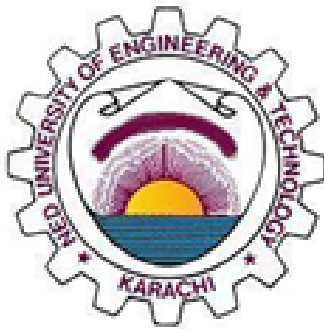


UNDERGRADUATE FINAL YEAR PROJECT REPORT

Department of Computer Science and Information Technology

NED University of Engineering and Technology



AI-Based Skin Lesion Analyzer with RAG-Based Self-Diagnosis Companion

Group Number: CS-22046

Batch: 2022-2026

Group Member Names:

Musfirah Saleem Shaikh

DT-22009

Shehroz Aijaz

DT-22011

Taroob Anwer

DT-22023

Syed Ezaan Haider Taqvi

DT-22038

Approved by

.....

Dr. Waseemullah
Associate Professor, CS & IT
Project Advisor



F/SOP/FYDP 02/06/00

Author's Declaration

We declare that we are the sole authors of this project. It is the actual copy of the project that was accepted by our advisor(s) including any necessary revisions. We also grant NED University of Engineering and Technology permission to reproduce and distribute electronic or paper copies of this project.

Musfirah Saleem Shaikh

DT-22009

shaikh4504235@cloud.neduet.edu.pk

Shehroz Aijaz

DT-22011

ejaz4506925@cloud.neduet.edu.pk

Taroob Anwer

DT-22023

anwer4520513@cloud.neduet.edu.pk

Syed Ezaan Haider Taqvi

DT-22038

taqvi4506499@cloud.neduet.edu.pk



Statement of Contributions

Musfirah Saleem Shaikh worked on the frontend interface of the image-classification module, as well as the RAG chatbot. She designed and developed initial layout structures making the interface clean, easy and user friendly. She completed end-to-end API integration, accessibility, and finished all user flows for image uploads, result visualization and interaction with a chatbot.

Shehroz Aijaz was the one to create the backbone of the ML/DL pipeline by collecting, pre-processing, training and optimizing the models. He designed and implemented CNN and hybrid ML-DL architectures, compared and contrasted them, and performed the training and fine-tuning life cycle for the ConvNeXt-Tiny model, and added the explainability module Grad-CAM into the prediction pipeline. Also, he led model evaluation and analysis of model performance.

Taroob Anwer helped collect, organize, and preprocess dermatology images dataset for the project. She helped with the evaluation, testing, and optimization of the models, and also worked on integrating the back-ends and validating the systems to ensure seamless communication between the AI models and application modules. She documented the FYDP and coordinated workflows in development.

Syed Ezaan Haider Taqvi was responsible for the Backend Development, Full system integration and implementing the RAG-based dermatology assistant. He has designed and implemented all FastAPI endpoints, created the database schema using SQLAlchemy ORM, built the auth module and security module and worked with all the sub systems to integrate them into a unified and deployable application. He also designed the RAG system architecture, built the vector database using pgvector, created the Parent-Child chunking strategy, and created the LangGraph orchestration workflow for multi-turn conversational AI.

All authors contributed equally to writing the final year project report.



Executive Summary

Skin diseases are one of the most prevalent health issues in the world and they need to be examined and treated properly when diagnosed and this is only possible with a timely visit to the dermatologists. Many problems do exist, though, for those seeking trustworthy advice from dermatologists: limited access to specialists, language barriers, and the wealth of untrusted medical information on the web. This work tackles these challenges by creating a skin disease screening platform with AI capabilities and a multilingual conversational support network.

The developed system is a complete web-based application that runs with a modular three-service architecture comprising a FastAPI backend service, a React based web front end, and a separate and isolated FastAPI machine learning inference service. Users can upload photos of skin to the platform and get AI-driven screening results for five types of skin conditions: Shingles, Chickenpox, Healthy Skin, Atopic Dermatitis, and Acne and Rosacea. The system also includes Grad-CAM heatmaps to make the model's prediction more explainable and user-trustworthy, showing the parts of the image that affect the model's prediction.

The platform also features an image classification system as well as a Retrieval-Augmented Generation (RAG) system with a dermatology guideline document library that enables grounded educational responses via a conversational assistant. The Chatbot can talk to the patient in English, Urdu, and Roman Urdu and adjusts its tone of interaction based on the patient and doctor mode. The patient mode emphasizes simple educational guidance, whereas doctor mode offers more technical and source-aware answers.

The machine learning pipeline was deployed with a fine-tuned ConvNeXt-Tiny model with the image preprocessing techniques, such as histogram equalization, image normalization, and augmentation techniques to make it robust to different image conditions. The backend features LangGraph orchestration, pgvector-based semantic search, JWT authentication, and PostgreSQL database management, enabling secure and persistent user interactions.

The frontend functionality, backend APIs, machine learning inference, multilingual chatbot behavior, security, and performance were all comprehensively tested and evaluated. The system had a good classification accuracy and conversational responsiveness with reasonable CPU latency in the deployment environment. The use of security features like bcrypt password hashing, JWT authentication, ownership filtering, and ORM-based database access ensured that user information was kept secure and inaccessible to unauthorized users.

The project showcases the ability to combine computer vision, conversational AI, and retrieval-augmented generation into a deployable health care support platform that is not necessarily for diagnosis, but rather, for educational dermatology support. The system is a multilingual explainable and modular AI solution for low resource environment and future expansion to telemedicine and mobile healthcare applications.



Acknowledgments

A special thanks to our project supervisor **Dr.Waseemullah** and co-supervisor **Dr.Usman Amjad** for his valuable guidance, continuous encouragement and constructive suggestions while preparing this final year project. His patience, understanding and positive approach were essential in developing our understanding and enhancing our work.

We also have a great debt of gratitude to our Industrial Advisor **Mr.Irtiza Hussain** (Senior Software Engineer, Folio3) for his invaluable knowledge and practical experience of the industry. His advice enabled us to connect the theory with practice, ensuring our project was according to professional standards and technological expectations in the real world. We would also like to immensely thank **Dr.Sana Adeel** (Dermatologist) for verifying our dataset and guiding us throughout this time.

We would also like to pay tribute to the dedication, ingenuity and determination of all group members. The idea was a successful, viable product as a result of each individual contribution.

Finally, we would like to thank all our faculty members, mentors and peers who supported, encouraged or inspired us during our journey. Your assistance, big or small, was a key part of the success of this project.



Contents

Author's Declaration	ii
Statement of Contributions	iii
Executive Summary	iv
Acknowledgments	v
List of Figures	x
List of Tables	xi
List of Abbreviations	xii
United Nations Sustainable Development Goals	xiv
Similarity Index Report	xv
1 Introduction	1
1.1 Background Information	1
1.2 Significance and Motivation	1
1.2.1 Improved access to derm support	1
1.2.2 Supporting Clinical Decision-Making	2
1.2.3 Increasing patient awareness and education	2
1.2.4 Contribution to Research and Innovation	2
1.2.5 Educational Value	2
1.3 Aims and Objectives	3
1.3.1 Aims	3
1.3.2 Objectives	3
1.3.3 Scope of the System	3
1.4 Methodology	4
1.4.1 Requirement Analysis	4
1.4.2 Preprocessing of Data Collected	4
1.4.3 Model Development and Training	4
1.4.4 RAG Pipeline Development	5
1.4.5 Frontend and Backend Integration	5
1.4.6 Testing and Evaluation	5
1.4.7 Documentation and Deployment	5
1.5 Report Outline	5



1.5.1	Chapter 1 — Introduction	5
1.5.2	Chapter 2 — Literature Review	5
1.5.3	Chapter 3 — Requirement Specification and Analysis	6
1.5.4	Chapter 4 — System Design and Implementation	6
1.5.5	Chapter 5 — Testing and Evaluation	6
1.5.6	Chapter 6 — Results and Discussion	6
1.5.7	Chapter 7 – Future Enhancements	6
1.5.8	Chapter 8 – Conclusion	6
2	Literature Review	7
2.1	Introduction	7
2.2	Skin Disease Burden and Need for AI	7
2.3	Traditional Skin Disease Diagnosis Methods	7
2.4	Classification of Skin Lesions using Deep Learning	8
2.5	Hybrid CNN-Transformer Architectures	8
2.6	Data Augmentation and Image Enhancement Techniques	8
2.6.1	Conversational AI in Healthcare	9
2.6.2	Retrieval-Augmented Generation (RAG) for medical systems	9
2.7	Existing Dermatology Applications and Systems	9
2.8	Summary of Reviewed Literature	10
2.9	Research Gaps	11
2.10	Summary	11
3	Requirement Specification and Analysis	12
3.1	Introduction	12
3.2	Roles and Characteristics of the users	12
3.2.1	End User / Patient	12
3.2.2	Clinician / Dermatologist	12
3.2.3	System Administrator	12
3.3	Functional Requirements	12
3.3.1	FR1-User Registration and Authentication (not mandatory in case of anonymity)	12
3.3.2	FR2 – Image Capture and Upload	13
3.3.3	FR3 – Image Pre-processing	13
3.3.4	FR4: Skin Disease Classification	13
3.3.5	FR5 – Result Display and Disclaimer	13
3.3.6	FR6 RAG-Based Chatbot Interaction	13
3.3.7	FR7 Management: Knowledge Base	14
3.3.8	FR8 – Logging and Analytics	14
3.3.9	FR9 – Error Handling	14
3.4	Non-Functional Requirements	14
3.4.1	NFR1 – Performance	14
3.4.2	NFR2 – Accuracy and Reliability	14
3.4.3	NFR3 – Scalability	15
3.4.4	NFR4 – Security and Privacy	15
3.4.5	NFR5 – Usability	15
3.4.6	NFR6 – Maintainability	15



3.4.7	NFR7 – Ethical and Legal Requirements	16
3.5	System Constraints and Assumptions	16
3.6	Preliminary technology stack (Indicative)	16
3.7	System Workflow	16
3.8	Use Case Diagram	17
3.9	System Architecture	19
3.10	Database Design / ERD	20
3.11	Summary	21
4	System Design and Implementation	22
4.1	Introduction	22
4.2	Overall System Workflow	22
4.3	Frontend Design and User Interface	22
4.4	Backend Architecture and API Integration	23
4.5	AI Model Development	23
4.5.1	Dataset Description	23
4.5.2	Data Preprocessing Techniques	24
4.5.3	Data Augmentation Techniques	24
4.5.4	Evaluated Deep Learning Models	25
4.5.5	ConvNeXt Architecture	26
4.5.6	Model Training Configuration	27
4.5.7	Performance Optimization Techniques	28
4.5.8	Final Model Selection	28
4.5.9	Image Processing and Prediction Pipeline	28
4.6	RAG-Based Medical Chatbot	29
4.6.1	Database Integration	29
4.6.2	Authentication and Security	30
4.6.3	System Interfaces and Screens	31
4.7	System Deployment	31
4.7.1	Target Deployment Environments	31
4.7.2	Local Deployment Using Docker Compose	32
4.7.3	Production Deployment on Google Cloud Platform	33
4.7.4	IAM Roles and Service Permissions	34
4.7.5	Environment Configuration	34
4.7.6	Scaling and Cold-Start Strategy	35
4.7.7	Networking and Security	35
4.7.8	Backup, Recovery, and Observability	36
4.7.9	Build and Rollout Process	36
4.7.10	Suitability of Cloud Run	36
4.8	System Interfaces and Screens	36
4.8.1	Landing Page and Authentication	36
4.8.2	Image Upload and Analysis Interface	37
4.8.3	Doctor Mode Dashboard	38
4.8.4	Patient Mode Dashboard	39
4.8.5	Multilingual Interaction	40
4.9	Summary	41



5	Testing and Evaluation	42
5.1	Introduction	42
5.2	Testing Strategy and Methodology	42
5.3	Frontend and Backend Testing	42
5.3.1	Frontend Testing	43
5.3.2	Backend Testing	44
5.3.3	API Integration Testing	45
5.4	AI Model Evaluation	45
5.4.1	Evaluation Metrics	45
5.4.2	Training and Validation Performance	46
5.4.3	Accuracy and Loss Analysis	46
5.4.4	Confusion Matrix Analysis	47
5.4.5	Comparative Model Analysis	48
5.4.6	Final Model Evaluation	48
5.5	Chatbot Evaluation	49
5.6	Performance and Security Testing	49
5.7	Results and Findings	49
5.8	Limitations	50
5.9	Summary	50
6	Results and Discussion	51
6.1	Overview of Results	51
6.2	Skin Disease Detection Performance	51
6.3	Chatbot Response Analysis	51
6.4	Comparative Analysis with Existing Systems	52
6.5	Challenges Faced During Development	52
6.6	Ethical and Medical Considerations	52
6.7	Summary	53
7	Future Enhancements	54
7.1	Introduction	54
7.2	Advanced Deep Learning Architectures	54
7.3	Improved RAG and Retrieval System	54
7.4	Multi-Language Support	54
7.5	Mobile Application Integration	55
7.6	Real-Time Clinical Integration	55
7.7	Summary	55
8	Conclusion	56
8.1	Project Summary	56
8.2	Achievements and Contributions	56
8.3	Lessons Learned	57
8.4	Final Remarks	57



List of Figures

1	SDLC Diagram	4
2	Data flow for Skensure	17
3	Use case diagram for Skensure actors and interactions	18
4	High-level architecture of the system	19
5	Entity-relationship-diagram for RAG	20
6	ConvNeXt Architecture	26
7	Local Deployment Using Docker Compose	33
8	Production Deployment Architecture on GCP	34
9	Production Deployment Architecture on GCP	35
10	Landing page and authentication screen showing role selection and secure login	37
11	Skin image uploaded and ready for analysis	37
12	Detected condition with confidence score and Grad-CAM explainability heatmap (Doctor Mode)	38
13	Doctor mode dashboard showing the skin scan panel and dermatology assistant with clinical prompts	39
14	Chatbot response in doctor mode with source document citation	39
15	Patient mode dashboard with simplified prompts and no clinical references . .	40
16	Chatbot responding in Urdu to a query about chickenpox symptoms	41
17	API Responses for testing backend	44
18	Evaluation Graphs for Convnext-tiny model	47
19	Confusion Matrix for Pretrained Resnet50(Left) and Convnext-tiny(Right) . .	47



List of Tables

1	Summary of Literature Review	10
2	Training Hyperparameters and Justifications	27
3	Supported Deployment Environments	32
4	IAM roles and service permissions.	34
5	Service scaling configuration.	35
6	Frontend testing results.	43
7	Backend testing results.	45
8	Classification Report Comparison: Pretrained ResNet50 vs ConvNeXt-tiny . .	48
9	Chatbot Evaluation Summary	49



List of Abbreviations

AAD	American Academy of Dermatology
AI	Artificial Intelligence
API	Application Programming Interface
BAD	British Association of Dermatologists
CLI	Command Line Interface
CNN	Convolutional Neural Network
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
DB	Database
DL	Deep Learning
Docker	Docker Containerization Platform
EHR	Electronic Health Record
EMR	Electronic Medical Record
FYDP	Final Year Design Project
GCP	Google Cloud Platform
GLCM	Grey-Level Co-occurrence Matrix
GPU	Graphics Processing Unit
Grad-CAM	Gradient-weighted Class Activation Mapping
HNSW	Hierarchical Navigable Small World
HTTP	Hypertext Transfer Protocol
ISIC	International Skin Imaging Collaboration
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning



NLP	Natural Language Processing
ORM	Object Relational Mapping
PCA	Principal Component Analysis
PDF	Portable Document Format
PII	Personally Identifiable Information
PNG	Portable Network Graphics
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RGB	Red Green Blue
RSS	Resident Set Size
SDG	Sustainable Development Goals
SMOTE	Synthetic Minority Over-sampling Technique
SPA	Single Page Application
SQL	Structured Query Language
SSE	Server-Sent Events
SVM	Support Vector Machine
TTFT	Time To First Token
UI	User Interface
UX	User Experience
ViT	Vision Transformer
XSS	Cross-Site Scripting
YAML	YAML Ain't Markup Language
YUV	Luminance Chrominance Color Space



United Nations Sustainable Development Goals

United Nations Sustainable Development Goals

The Sustainable Development Goals (SDGs) are the blueprint to achieve a better and more sustainable future for all. They address the global challenges we face, including poverty, inequality, climate change, environmental degradation, peace, and justice. There are a total of 17 SDGs as mentioned below.

Check the appropriate SDGs related to the project.

- ☐ No Poverty
- ☐ Zero Hunger
- ✓ Good Health and Well-being
- ☐ Quality Education
- ☐ Gender Equality
- ☐ Clean Water and Sanitation
- ☐ Affordable and Clean Energy
- ☐ Decent Work and Economic Growth
- ✓ Industry, Innovation, and Infrastructure
- ✓ Reduced Inequalities
- ☐ Sustainable Cities and Communities
- ☐ Responsible Consumption and Production
- ☐ Climate Action
- ☐ Life Below Water
- ☐ Life on Land
- ☐ Peace and Justice and Strong Institutions
- ☐ Partnerships to Achieve the Goals



Similarity Index Report

Following students have compiled the final year report on the topic given below for partial fulfillment of the requirement for Bachelor's degree in Computer Science & Information Technology.

Project Title: AI-Based Skin Lesion Analyzer with RAG-Based Self-Diagnosis Companion
AI-Based Skin Lesion Analyzer with RAG-Based Self-Diagnosis Companion

S. No.	Student Name	Seat Number
1.	Musfirah Saleem Shaikh	DT-22009
2.	Shehroz Aijaz	DT-22011
3.	Taroob Anwer	DT-22023
4.	Syed Ezaan Haider Taqvi	DT-22038

This is to certify that Plagiarism test was conducted on the complete report, and overall similarity index was found to be less than 20%, with a maximum 5% from a single source, as required.

Signature and Date

Dr. Waseemullah

[Plagiarism Test must be conducted on the project report by the project advisor using Turnitin Software, or any other software approved by the university. Project Advisor is required to sign the Plagiarism Certificate if the similarity is found as per rules defined in the guidelines. Plagiarism report must be included (only the first and last page) generated by Turnitin. A sample of the plagiarism test report is provided with the FYDP template package.]

[Attach here the first and last page of the plagiarism report generated by Turnitin.]

10% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

-  **166 Not Cited or Quoted 9%**
Matches with neither in-text citation nor quotation marks
-  **7 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 6%  Internet sources
- 2%  Publications
- 9%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 166 Not Cited or Quoted 9%**
Matches with neither in-text citation nor quotation marks
- 7 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 6% Internet sources
- 2% Publications
- 9% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

- 1** Student papers
Higher Education Commission Pakistan on 2022-08-04 <1%
- 2** Student papers
Higher Education Commission Pakistan on 2022-08-01 <1%
- 3** Internet
www.mdpi.com <1%
- 4** Student papers
St Joseph's Regional College on 2025-05-20 <1%
- 5** Student papers
St.Xavier's Catholic College of Engineering on 2026-04-22 <1%
- 6** Internet
portalcris.vdu.lt <1%
- 7** Internet
se.neduet.edu.pk <1%
- 8** Internet
www.coursehero.com <1%
- 9** Student papers
INTI Universal Holdings SDM BHD on 2026-04-24 <1%
- 10** Student papers
University of Sydney on 2024-10-12 <1%

11	Student papers	7034 on 2015-03-20	<1%
12	Student papers	Higher Education Commission Pakistan on 2023-02-16	<1%
13	Student papers	Strathmore University (Main Account) on 2026-03-13	<1%
14	Student papers	Strathmore University (Main Account) on 2026-03-18	<1%
15	Publication	A. Punitha, S. Syedakbar, S. Jeyasudha. "Intelligent and Sustainable Systems: AI, G...	<1%
16	Student papers	University of Greenwich on 2026-04-20	<1%
17	Publication	B. G. Nagaraja, S. Kannadhasan. "Information and Communication Systems", CRC...	<1%
18	Student papers	Higher Education Commission Pakistan on 2022-07-27	<1%
19	Student papers	Islington College,Nepal on 2026-04-22	<1%
20	Student papers	University of Hong Kong on 2026-04-21	<1%
21	Student papers	Liverpool John Moores University on 2025-04-12	<1%
22	Internet	f1000research-files.f1000.com	<1%
23	Student papers	Zambia Centre for Accountancy Studies on 2026-06-12	<1%
24	Internet	www.scribd.com	<1%

25	Student papers	Sunway Education Group on 2024-06-21	<1%
26	Internet	dl.ucsc.cmb.ac.lk	<1%
27	Internet	user.ceng.metu.edu.tr	<1%
28	Student papers	Asia Pacific Institute of Information Technology on 2025-08-26	<1%
29	Student papers	Australian Catholic University on 2026-06-08	<1%
30	Publication	"Innovations in Data Analytics", Springer Science and Business Media LLC, 2025	<1%
31	Internet	123docz.net	<1%
32	Student papers	British University in Egypt on 2026-06-03	<1%
33	Student papers	Higher Education Commission Pakistan on 2023-02-19	<1%
34	Student papers	K L University on 2025-02-21	<1%
35	Student papers	De Montfort University on 2026-05-28	<1%
36	Student papers	Sydney Institute of Technology and Commerce on 2026-05-31	<1%
37	Student papers	City University on 2024-10-01	<1%
38	Student papers	Endeavour College of Natural Health on 2026-05-10	<1%

39	Internet	ebin.pub	<1%
40	Internet	www.frontiersin.org	<1%
41	Student papers	Amrita Vishwa Vidyapeetham on 2026-06-08	<1%
42	Student papers	Erasmus University of Rotterdam on 2021-05-09	<1%
43	Student papers	Vellore Institute of Technology on 2026-03-26	<1%
44	Internet	graduate.gradsch.uga.edu	<1%
45	Student papers	Islington College,Nepal on 2025-12-30	<1%
46	Student papers	Southampton Solent University on 2026-01-05	<1%
47	Student papers	University of Portsmouth on 2026-05-20	<1%
48	Internet	www.perthdermatologyclinic.com.au	<1%
49	Student papers	Technische Hochschule Deggendorf on 2026-04-29	<1%
50	Student papers	University of Hertfordshire on 2026-01-04	<1%
51	Student papers	University of Newcastle on 2023-10-31	<1%
52	Internet	digitalcommons.unl.edu	<1%

53	Internet	www.ijrti.org	<1%
54	Student papers	Higher Education Commission Pakistan on 2022-08-22	<1%
55	Student papers	Liverpool John Moores University on 2022-08-17	<1%
56	Student papers	Technological Institute of the Philippines on 2024-07-02	<1%
57	Student papers	University of Melbourne on 2020-11-12	<1%
58	Internet	addaxdatascience.com	<1%
59	Internet	conservancy.umn.edu	<1%
60	Internet	humantranslator.blogspot.com	<1%
61	Internet	mdpi-res.com	<1%
62	Internet	pmc.ncbi.nlm.nih.gov	<1%
63	Student papers	Albukhary International University on 2025-09-23	<1%
64	Student papers	Glasgow Caledonian University on 2025-05-23	<1%
65	Student papers	Southampton Solent University on 2025-01-11	<1%
66	Student papers	Swinburne University of Technology on 2024-12-08	<1%

67	Internet	assets-eu.researchsquare.com	<1%
68	Internet	gs.elaba.lt	<1%
69	Internet	journal.unm.ac.id	<1%
70	Internet	wagga.nsw.gov.au	<1%
71	Internet	www.shine.com	<1%
72	Student papers	Asia Pacific Institute of Information Technology on 2026-02-13	<1%
73	Publication	Oluwayemisi Jaiyeoba, Emeka Ogbuju, Grace Ataguba, Oluwaseyi Jaiyeoba, James...	<1%
74	Student papers	SRM University on 2025-04-25	<1%
75	Student papers	Swinburne University of Technology on 2024-12-05	<1%
76	Student papers	Turku University of Applied Sciences on 2026-06-10	<1%
77	Student papers	University of East London on 2026-05-07	<1%
78	Student papers	University of Hertfordshire on 2010-03-15	<1%
79	Student papers	University of Surrey on 2023-05-15	<1%
80	Student papers	University of Westminster on 2025-04-01	<1%

81	Student papers	University of Westminster on 2025-05-15	<1%
82	Student papers	Vellore Institute of Technology on 2026-04-19	<1%
83	Publication	de Almeida, Gonalo Fernandes Diogo. "Real-Time Detection of Driver Fatigue Usi...	<1%
84	Internet	dspace.daffodilvarsity.edu.bd:8080	<1%
85	Internet	file.techscience.com	<1%
86	Student papers	Asia Pacific Institute of Information Technology on 2026-02-13	<1%
87	Student papers	Berlin School of Business and Innovation on 2024-06-06	<1%
88	Student papers	Dawood University of Engineering & Technology, Karachi on 2026-05-13	<1%
89	Publication	Dwane Joseph Joel, Anshuman Sharma, Mithun Kanchan, Omkar S Powar. "Deep L...	<1%
90	Student papers	Higher Education Commission Pakistan on 2023-07-23	<1%
91	Student papers	Hong Kong Baptist University on 2020-07-02	<1%
92	Student papers	Liverpool John Moores University on 2025-08-04	<1%
93	Student papers	Midlands State University on 2015-05-07	<1%
94	Student papers	Staffordshire University on 2024-05-13	<1%

95	Student papers	University of Bedfordshire on 2018-03-09	<1%
96	Student papers	University of Technology, Sydney on 2024-11-06	<1%
97	Student papers	University of Ulster on 2023-05-07	<1%
98	Student papers	University of Ulster on 2026-04-17	<1%
99	Internet	arccjournals.com	<1%
100	Internet	arxiv.org	<1%
101	Internet	dom.lndb.lv	<1%
102	Internet	dspace5.zcu.cz	<1%
103	Internet	eprints.soton.ac.uk	<1%
104	Internet	export.arxiv.org	<1%
105	Internet	proceedings.neurips.cc	<1%
106	Internet	repositorio-iul.iscte.pt	<1%
107	Internet	www.iaeng.org	<1%
108	Publication	Silva, José Hugo Sousa. "Innovative Solutions for Healthcare Data Management: T...	<1%

109	Student papers	Thesis 2025 on 2025-12-29	<1%
110	Student papers	Albukhary International University on 2026-01-23	<1%
111	Publication	Bhagyashri S. Sonune, R. Udaykumar, Shon G. Nemane, Dhiraj P. Tulaskar, Manis...	<1%
112	Student papers	Islington College,Nepal on 2026-04-22	<1%
113	Student papers	Manchester Metropolitan University on 2025-09-15	<1%
114	Student papers	Manchester Metropolitan University on 2026-05-10	<1%
115	Student papers	University of Ulster on 2025-12-12	<1%
116	Student papers	University of the Philippines Los Banos on 2026-05-28	<1%

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.



Chapter 1

Introduction

1.1 Background Information

Skin diseases were one of the most important global health problem, and chronic inflammatory skin diseases like atopic dermatitis, acne, chickenpox and shingles were among the leading causes of morbidity and decreased quality of life. They would be seen clearly and often had a social stigma attached with them and would need repeated consultation and long treatment. There was lack of dermatologists and patient had to wait for a long time for a proper diagnosis in Pakistan, similar to other South Asian countries, where the patients self-treated and complications arose.

At the same time, ubiquitous smartphone uptake and readily available low cost cameras allowed for skin image capture to be easy, even outside of clinical environments. Decision support is a natural process of computer vision for dermatology which was a predominantly visual domain. Deep learning and convolutional neural networks (CNNs) were equivalent to or surpassed performance of human experts in the same study on several non-malignant skin diseases in large annotated image datasets.

Conversational AI quickly made its way into healthcare alongside image analysis. Medical chatbots and virtual assistants were used for triage, patient education and follow up. The main idea of the proposed system, known as Retrieval-Augmented Generation (RAG), was to leverage a large language model (LLM) in conjunction with an external, trusted knowledge base to produce accurate and explainable responses that draw on medical literature and not the black-box knowledge of the LLM.

The two trends of this project were AI-powered image-based screening of common skin diseases along with explainable AI to explain and give patients safe, accessible, and understandable explanations and self-care instructions, and a RAG-based dermatology assistant to give patients a safe, understandable and instructional explanation along with self-care guidance.

1.2 Significance and Motivation

1.2.1 Improved access to derm support

Access to dermatology services remains a significant problem, especially in low and middle resource settings, where dermatologists are more likely to be located in urban areas. Those who live in rural or underserved areas often have to wait longer for a consultation, travel greater distances to get care, or wait longer for medical attention. The restrictions have led to



people self-diagnosing with information from unreliable sources online, via social media and non-medical websites, often leading to misinformation and inappropriate treatment choices. The rationale for this project was to offer a more accessible and technology-driven solution that aids to early detection of common skin conditions. The AI-driven image classification, coupled with the conversational support, aims to make the system more user-friendly, helping users better understand the potential skin condition and identify when to seek professional medical attention.

1.2.2 Supporting Clinical Decision-Making

The other great driving idea behind the project was the rising demand for decision-support tools in telemedicine and overall health care. Skin disease diagnosis is something that may be encountered daily by general practitioners and non-specialist health care providers, but without advanced expertise. The system proposed makes an additional contribution by supporting the healthcare professionals for probable predictions and highlighting those cases which might need a specialist referral. The use of explainable AI methods enhances trust and transparency by helping users and practitioners understand why the model is making a prediction rather than just taking its results on faith. This renders the system more applicable to practical care and initial triage in the healthcare system.

1.2.3 Increasing patient awareness and education

Patient education is still a critical issue in dermatology, especially with brief consultations and few possibilities of follow-up. Lots of patients have poor knowledge of the triggers, self-care, warning signs and long-term management of their diseases. Barriers to the availability of reliable health information also include language and literacy barriers. The problem tackled in this project is making the chatbot understandable, context-aware and multilingual by implementing the Retrieval-Augmented Generation (RAG) framework. The system facilitates communication in both English and Urdu, including Roman Urdu, making it more accessible for South Asian users and fostering awareness about skin health and adherence to treatment.

1.2.4 Contribution to Research and Innovation

The project is also an important contribution to the research of AI-assisted healthcare. Many studies have addressed the single classification of images, or conversational health systems without combining the two functions. In this project, we take a step to fill the gap between deep learning based diagnosis and a knowledge-based dermatology chatbot. Further, hybrid ML-DL architectures are explored and the performance is assessed for various skin conditions and different skin tones. This helps address some of the challenges of fairness, generalization and reliability that remain with research in AI applications for dermatology.

1.2.5 Educational Value

The system is not only patient assistance but a valuable learning tool for medical students and junior doctors. The classification of images is useful for dermatology pattern recognition and learning the difference between various patterns and diseases, and the chatbot has good skills



in communicating and explaining to patients. The platform therefore could be useful in both the academic and clinical training setting.

1.3 Aims and Objectives

1.3.1 Aims

This project aimed to create an end-to-end AI health assistant system for dermatology, integrating an AI-based image classification model with a retrieval-augmented conversational agent to aid initial assessments of skin diseases and patient education.

1.3.2 Objectives

The project focused on achieving several technical and functional objectives:

- To curate and preprocess a clean multi-class dermatological image dataset of five common non-malignant skin conditions.
- To develop and test several deep learning architectures and choose the most accurate model that is explainable by the clinician.
- To enable Grad-CAM visual explainability of transparent and interpretable AI predictions.
- To create a multilingual Retrieval-Augmented Generation (RAG) based chatbot using LangGraph and a set of curated PDF resources related to dermatology.
- To create a dual-mode user interface for both patients and doctors, with React 18.
- To integrate all the frontend, backend, machine learning, and chatbot subsystems into a secure FastAPI backend, using the PostgreSQL database for persistence.

1.3.3 Scope of the System

The scope of this project was defined as follows:

- The system was able to classify five types of skin conditions (Vi Shingles, Vi Chickenpox, Healthy, Acne and Rosacea, and Atopic Dermatitis).
- The chatbot was restricted to answer the questions based on the provided medical PDF corpus and did not reach the internet in real time.
- The system was specifically designed as a learning education screening and triage tool, explicitly not a clinical diagnostic device.
- Multilingual support was provided for English, Urdu (in Urdu script), and Roman Urdu (Hinglish).
- The platform targeted web and mobile browser environments; a native mobile application was outside scope.

1.4 Methodology

The project was designed, developed, integrated, and evaluated systematically using a structured Software Development Life Cycle (SDLC) methodology to ensure systematic design, development, integration, and evaluation of the project. The methodology integrated machine learning development, conversational AI implementation, full-stack web application engineering, and system testing in one stream. Every stage of the project was meticulously designed to guarantee the reliability of the model's performance, seamless integration with the backend, ease of access for users, and readiness for practical use.

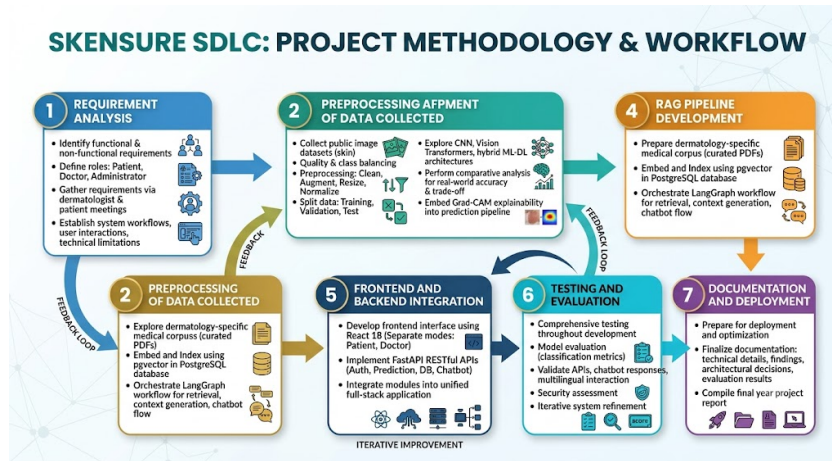


Figure 1: SDLC Diagram

1.4.1 Requirement Analysis

The development process started with the analysis of requirements, which covered the identification of the functional and non-functional requirements for each role—that is, patient, doctor, and medical students. This was done by meeting dermatologists and their patients. The project foundation, in terms of system workflows, user interactions, and technical limitations, was defined.

1.4.2 Preprocessing of Data Collected

Image datasets of Shingles, Chickenpox, Healthy Skin, Atopic Dermatitis, and Acne Rosacea were gathered through public access sources such as Dermnet.com and Dermatology Atlas which were then checked for quality and class balance. To enhance the performance and generalization of the model, the images were preprocessed, cleaned, augmented, resized, normalized, and then split into training, validation, and test sets.

1.4.3 Model Development and Training

Several convolutional neural network (CNN), vision transformer, and hybrid ML-DL architectures were explored and tested. After comparative analysis, the architecture with the highest real-world accuracy and the best trade-off was selected. Grad-CAM explainability was embedded in the prediction pipeline to visually show which parts of the image affected the model prediction.



1.4.4 RAG Pipeline Development

A dermatology-specific medical corpus was prepared from curated PDF resources. The documents were embedded and indexed using pgvector within a PostgreSQL database. A Lang-Graph orchestration workflow was developed to manage retrieval, context generation, and conversational response flow within the RAG chatbot pipeline.

1.4.5 Frontend and Backend Integration

The frontend interface was developed using React 18 with separate workflows for patient and doctor modes. FastAPI was used to implement RESTful APIs for authentication, image prediction, database communication, and chatbot interaction. All modules were integrated into a unified full-stack application.

1.4.6 Testing and Evaluation

Comprehensive testing was performed throughout development. This included model evaluation using classification metrics, API validation, chatbot response testing, multilingual interaction testing, and security assessment. The system was refined iteratively based on evaluation outcomes.

1.4.7 Documentation and Deployment

The final stage involved deployment preparation, optimization, and documentation. Technical implementation details, experimental findings, architectural decisions, and evaluation results were compiled into the final year project report.

1.5 Report Outline

1.5.1 Chapter 1 — Introduction

The first chapter will offer background and the significance of AI-assisted dermatology systems while addressing issues caused by the lack of access to dermatological guidance, unreliable information sources, and the language barrier. In this chapter, the problem statement, goals and objectives of the project, scope of the project, solution, methodological framework, and structure of the paper will be presented.

1.5.2 Chapter 2 — Literature Review

The second chapter will provide the literature review of previous works regarding the diagnosis of skin diseases using ML and DL methods, Explainable AI, hybrid models using CNN and transformers, conversational AI systems, and Retrieval-Augmented Generation for the health care industry. This chapter will examine existing dermatology applications and their limitations and will describe how the current project resolves them.



1.5.3 Chapter 3 — Requirement Specification and Analysis

This chapter will identify the functional and non-functional requirements of the system under development. It will explore roles of stakeholders, describe modules and use cases, discuss workflows, provide activity diagrams and architectural planning. It will also explain how requirements of the users were turned into software components and system functionality.

1.5.4 Chapter 4 — System Design and Implementation

In this chapter, design and implementation of the built platform will be described. It will provide information about frontend and backend architecture, inference engine of the AI model, data storage, API calls, and deployment structure. Details about the implementation of the ConvNeXt-Tiny classifier, Grad-CAM explainability module, multi-lingual chatbot, Lang-Graph orchestration framework, and RAG retrieval engine will be provided.

1.5.5 Chapter 5 — Testing and Evaluation

This chapter will describe the testing and evaluation methods that were applied to validate the system. Frontend and backend integration tests, AI models evaluation, chatbot test, performance testing, security testing, and validation of the user workflow will be covered. Potential limitations found during the testing will be discussed along with the effectiveness of the implemented solution.

1.5.6 Chapter 6 — Results and Discussion

This chapter will present the results and discussion of the implemented platform. Performance of skin disease detection algorithm, behavior of the chatbot, multilingual chatbot interaction, and effectiveness of the system as a whole will be analyzed. The platform will be compared to other available solutions, and technical difficulties and ethical issues will be addressed.

1.5.7 Chapter 7 – Future Enhancements

In this chapter, possible enhancements and extensions for the platform are outlined. They involve developments related to neural networks' architecture design, possible improvements in retrieval systems, new languages' implementation, and possibilities of using the platform with mobile devices. In addition, future directions may be concerned with incorporating the platform into clinical procedures.

1.5.8 Chapter 8 – Conclusion

The purpose of this chapter is to summarize all results that have been achieved during the course of the research and development project. Furthermore, it evaluates the level of achievement of goals set by the project and identifies whether the problem has been solved through the platform construction. Finally, importance of explainable and multilingual AI-powered healthcare solutions will be discussed.



Chapter 2

Literature Review

2.1 Introduction

This chapter summarizes previous literature in three main areas: image-based Explainable machine learning and deep learning for diagnosis of skin diseases and conversational AI and RAG-based systems in healthcare. The review was directed towards skin conditions and algorithms that are applicable in internet system component deployments in resource constrained environments, directly influencing the design decisions in this project.

2.2 Skin Disease Burden and Need for AI

It was found from the epidemiological literature that skin diseases were a significant proportion of the non-fatal disease burden globally, with chronic inflammatory and infectious diseases accounting for a major share of outpatient visits (Zhang et al., 2023). A great many skin diseases were very stigmatising, symptomatic and conspicuous, resulting in anxiety and depression, and dermatology. Most of the health systems failed to provide adequate service to remained underserved (Du-Harpur et al., 2020; Mahil et al., 2024).

systematic review was performed by Choy et al. (2023) to analyze the use of deep learning applications across 144 types of skin diseases and noted that researchers concentrated on common inflammatory conditions such as acne, chicken pox, eczema and rosacea. Deep learning algorithms achieved high accuracy in the diagnosis of vitiligo, and urticaria. Diagnostic accuracy is usually over 90%. But the authors wrote that they had expressed concerns about bias, incomplete reporting of skin types and lack of external validation (Choy et al., 2023). These restrictions prompted the creation of systems that are tailored for a more diverse audience.

2.3 Traditional Skin Disease Diagnosis Methods

In this section, we'll examine some traditional methods used to diagnose skin diseases. The first skin disease detectors were based on manual features and classical algorithms of machine learning. Typical pipelines divided the region of the lesion, and extracted it the features used for colour, shape and texture based on grey-level Co-occurrence Matrix (GLCM).

K-Nearest Neighbor, Support Vector Machines, and Decision Trees classifiers were applied. Such methods needed hand-made feature engineering or often didn't support changes to illumination, heterogeneous background, and complex lesion morphologies (Bajwa et al., 2020; Du-Harpur et al., 2020).



2.4 Classification of Skin Lesions using Deep Learning

Recent research focused on convolutional neural networks (CNNs). The hierarchical feature representations were automatically extracted from the pixels data (Dang et al., 2021; Liu et al., 2020). Standard pipelines included image acquisition, Aggressive pre-processing such as size reduction, denoising and contrast enhancement. The augmentation techniques applied to the data consisted of mirroring, blurring, and adding noise, while the classification methods utilized were CNN with accuracy. The precision, recall, and f1 score are used to measure model performance (Choy et al., 2023; Liu et al., 2020).

According to Choy et al. (2023), about 80% of the deep learning components they included in their analysis were found to be outdated. CNN or deep CNN models were used in the studies, while multiclass classification. In the case of the most common configuration, mediocre accuracy around 94% was reached for most. For acne and rosacea, 94%; for eczema, 89%; for psoriasis, 89%. More recent work by The use of ResNet50 with EfficientNet has been shown to be effective by Alruwaili and Mohamed. variants scored excellent accuracy rates for various inflammatory, infectious, and pigmentary skin diseases.

2.5 Hybrid CNN-Transformer Architectures

Several works suggested hybrid models of deep feature extractors and shallow classifiers. To meet the requirements of accuracy, interpretability, and computational efficiency, classical ML classifiers are used. Transfer learning was Aquil et al. (2025) applied to MobileNetV2, DenseNet121, and EfficientNetB0 which were used to extract dermoscopic images. These features were then passed on to SVM, Random Forest, and Decision Tree algorithms. For class, the classifiers performed well with accuracy values around 98%, when SMOTE was applied. Two types of methods were used: SMOTE for class imbalance and PCA for dimensionality reduction.

A second line of work Dang et al. (2021) fused lightweight CNN backbones like MobileNetV2 with LSTM modules to learn the relationship in higher level feature. These maps are also enriched with texture features from GLCM, and are rendered in a natural human style. These studies suggested evolution of lightweight CNNs backbones and shallow classical classifiers. Sensible design option for limited resource deployments.

2.6 Data Augmentation and Image Enhancement Techniques

This section discusses some of the data augmentation and image improvement techniques. This section covers some of the data augmentation and image improvement techniques.

Data augmentation is known to be crucial in medical image analysis. The relative dearth of labeled dermatologic data. Random flips, rotations, There are several commonly used techniques to deal with class imbalance, including and colour jitter, and others tackled such as SMOTE. Early detection system(s) based on class imbalance at the feature level (Aquil et al., 2025). Preprocessing the image was then given a contrast stretch using histogram equalisation



in the YUV colour space, followed by a Gaussian filtering. We have shown that normalisation using ImageNet and blurring significantly stabilize. To reduce the effects of uncontrolled clinical lighting conditions during training. (Bajwa et al., 2020; Zhang et al., 2023).

2.6.1 Conversational AI in Healthcare

In the field of dermatology, conversational AI became a growing field of study, and with a wide range of pre-screening symptom collection and basic patient characteristics features. Information and education on skin conditions, reminders for treatment compliance, and triage assistance. (Chen et al., 2023). Literature on dermatology chatbots highlighted that although AI could improve access and patient engagement, deployment will take transparency. Disclaimers, bias reduction measures and clinician oversight (Patel et al., 2021). Initial deployment reports indicated patients were found Empathic, clear responses given by AI when agents ask questions focused on information support instead of being carefully supervised were autonomous diagnosis (Gilson et al., 2023; Sarrouiti & El Alaoui, 2024).

2.6.2 Retrieval-Augmented Generation (RAG) for medical systems

LLMs, especially large-scale models, had the tendency to produce coherent, but factually incorrect, responses. incorrect medical statements. To overcome this shortcoming, RAG founded a new approach to model. Responses in confirmed domain-specific documents at inference time Ramoliya (2024) and Russo (2024) used RAG to apply it to. GPT-4 to create a medical chatbot that was contextually aware and had content grounded in medicine. GPT-4 to generate a medical chatbot with context and medical knowledge. in clinical guidelines, showing a higher degree of factual correctness than standard LLM prompting showed that open-source LLMs can effectively handle patient care tasks. The open-source LLMs in standard LLM prompting were shown to be effective in patient care tasks by (Ramoliya, 2024). LLMs combined with Qdrant vector databases could be utilized for local, privacy-preserving deployments.

In the dermatology field, a chatbot that is equipped with RAG could index dermatology reviews to articles, patient-education leaflets, national guidelines to produce valid, natural-language answers (directly addressing).

2.7 Existing Dermatology Applications and Systems

In this section, existing dermatology applications and systems will be described. Existing dermatology applications and systems will be described in this section.

The current AI dermatology models were typically categorized into two types: image-only and image-text. The AI dermatology models that already existed were mainly in two categories: image-only and image-text. classifiers that did not use a conversational layer or have a conversational layer that was not used in the prediction output, and or to general purpose medical chatbots that do not have imaging diagnostic features. No widely-used system combined an image classifier and a domain-specific Common non-malignant dermatological conditions RAG chatbot. Russo (2024) and Ramoliya (2024) gave architectural guidance. The network



structure for the RAG pipeline is inspired from the ResNet-based model and ConvNeXt-based model. The most recent literature includes classification systems (Alruwaili & Mohamed, 2025; Aquil et al., 2025) that informed the model selection process.

2.8 Summary of Reviewed Literature

Table 1 presents a consolidated overview of the key works reviewed in this chapter. Each entry highlights the domain focus, the techniques and insights drawn from the study, and their direct relevance to the design decisions made in this project. Together, these works establish the theoretical and empirical foundation upon which this project is built.

Table 1: Summary of Literature Review

Author(s)	Year	Domain Focus	Techniques / Insights Learned	Techniques Used in Our Project
Alruwaili & Mohamed (2025)	2025	Multi-class skin disease classification	Hybrid deep learning using EfficientNet + ResNet; feature fusion; high accuracy benchmarks	Understanding modern CNN backbones; justification for using ResNet-based feature extraction
Bajwa et al. (2020)	2020	Deep learning for dermatology	CNN-based diagnosis; importance of large datasets; end-to-end deep models	CNN model selection guidance; supports using image-based deep learning
Chen et al. (2023)	2023	Dermatology chatbots	Risks & benefits of consumer-facing AI; safe output guidelines	Helps define chatbot safety rules & disclaimers
Choy et al. (2023)	2023	Systematic DL review	Performance ranges for acne, eczema, psoriasis; fairness limitations; dataset biases	Justifies using non-malignant classes; importance of diverse skin-tone dataset
Daneshjou et al. (2022)	2022	Fairness in AI dermatology	Bias detection; evaluating performance across skin tones	Motivation for balanced data & meta-data inclusion
Dang et al. (2021)	2021	Hybrid DL interpretability	Feature fusion; interpretable DL; hybrid CNN-ML pipeline	Supports using hybrid CNN + ML classifier (e.g., ResNet features + SVM)
Aquil et al. (2025)	2025	Hybrid ML–DL	MobileNetV2, DenseNet121, EfficientNetB0 + SVM/RF/DT; PCA + SMOTE	Guidance for hybrid models; validates combining deep features with classical ML
Du-Harpur et al. (2020)	2020	AI in dermatology	Overview of AI applications in dermatology; key clinical considerations	High-level justification for using AI for skin diagnosis
Gilson et al. (2023)	2023	Medical LLM evaluation	Benefits & risks of GPT-4-powered AI assistants; hallucination control	Supports the decision to use RAG rather than raw LLM output
Russo (2024)	2024	RAG implementation	Real-world RAG pipeline implementation; combining retrieval + LLM	Architecture inspiration for our RAG chatbot
Liu et al. (2020)	2020	Deep differential diagnosis	Large multi-class DL system; metadata improves accuracy	Motivates adding optional metadata fields (duration, body location)
Mahil et al. (2024)	2024	Skin disease DL landscape	Review of DL performance across disease ranges; clinical relevance	Helps justify common dermatology classes for training
Ramoliya (2024)	2024	RAG-based Medical Chatbot	Qdrant Vector Database, Retrieval-Augmented Generation (RAG), LLM Integration, Medical Knowledge Retrieval	Demonstrates domain-specific medical question answering using RAG; provides practical implementation of vector search and context-aware response generation
Patel et al. (2021)	2021	AI for dermatologists	AI support tools in dermatology; clinical triage role	Motivates positioning system as decision-support, not diagnosis
Sarrouti & El Alaoui (2024)	2024	LLMs & RAG	Impact of ChatGPT; advantages of retrieval grounding	Reinforces decision to use RAG instead of normal chatbot
Zhang et al. (2023)	2023	ML/DL review	GLCM, MobileNet, ResNet, EfficientNet; recent DL trends	Supports hybrid feature extraction + CNN approach



2.9 Research Gaps

Based on the review above, several gaps and opportunities were identified:

- **Non-malignant Emphasis:** Most high-profile works concentrated on malignancy detection, whereas few systems addressed common conditions such as eczema, acne, and infectious rashes that constituted the majority of dermatology consultations (Choy et al., 2023; Zhang et al., 2023).
- **Deployment in Low-Resource Settings:** Few models were optimised for low- and middle-income environments that required lightweight, efficient inference on limited hardware (Aquil et al., 2025; Bajwa et al., 2020).
- **Integrated Educational Layer:** Most image-based studies concluded with classification and lacked a conversational interface to inform and educate patients (Chen et al., 2023; Patel et al., 2021).
- **RAG-Based Dermatology Assistants:** General medical chatbots existed, but there was limited research integrating RAG with a dermatology image classifier (Ramoliya, 2024; Russo, 2024).
- **Fairness and Transparency:** Few well-annotated, diverse multi-skin-tone datasets existed, and model behaviour across demographic subgroups was rarely reported (Daneshjou et al., 2022; Mahil et al., 2024).

2.10 Summary

This chapter identified the major gaps and opportunities in the existing dermatology-oriented AI systems. Most of the solutions available have concentrated on: The current tools for cancer detection were limited to rare instances of everyday dermoscopy and few tools for the diagnosis of common everyday skin lesions were developed. conditions. There were multiple models that could not be optimised for low resource deployments, Many image-based tools and none of the image based tools had an educational or conversational overlay. The The use of RAG in combination with dermatology image classifiers was still uncommon. (Ramoliya, 2024; Russo, 2024). These were directly addressed in this project. To produce a light weight non-cancer specific classifier along with a score. Multilingual dermatology assistant that is able to identify typical conditions based on RAG Inflammatory and infectious skin diseases are common conditions that can be seen in the skin (Alruwaili & Mohamed, 2025; Choy et al., 2023).

Chapter 3

Requirement Specification and Analysis

3.1 Introduction

The problem definition and literature understanding is converted into a framework of functional and non-functional requirements of the AI Skin Disease Diagnosis and RAG Chatbot System in this chapter. It also establishes meaningful user roles and system constraints and assumptions that will be utilized during further design and implementation.

3.2 Roles and Characteristics of the users

3.2.1 End User / Patient

- Little knowledge of medicine, if any but not a medical expert.
- Mobile surfers (with smartphone or web browser).
- Need basic interface, warning messages and language (English and Urdu).

3.2.2 Clinician / Dermatologist

- May examine prediction and heatmap of the system.
- Are interested in model confidence and model explanation.

3.2.3 System Administrator

- Hands on user account management, data and content management of knowledge base.
- Keeps track of logs and performance.

3.3 Functional Requirements

3.3.1 FR1-User Registration and Authentication (not mandatory in case of anonymity)

System will make it possible for users to register and log-in using an email or phone; or provide an anonymous option using image processing consent.



3.3.2 FR2 – Image Capture and Upload

System will enable users to:

- Set up a photo gallery – choose an existing skin image, or
- Take a new picture with the in-built camera.

3.3.3 FR3 – Image Pre-processing

System shall automatically:

- Resamp size to input resolution of the model,
- Normalize pixel values,
- Simple removal of artefacts (e.g. cropping of border).

3.3.4 FR4: Skin Disease Classification

When an image (and optional metadata) is presented to it, the system shall:

- Run the CNN / hybrid ML–DL model,
- Make the k most likely disease classes (e.g. top-3),
- Give related scores of confidence (probabilities).

3.3.5 FR5 – Result Display and Disclaimer

The UI shall display:

- Disease label(s),
- Confidence scores,
- Explanation of the situation in brief,
- Self-management and common management,
- Warning that it is not a clinical diagnosis, and that users should see a dermatologist, especially if symptoms are becoming more severe.

3.3.6 FR6 RAG-Based Chatbot Interaction

After predictions, the users will be able to open a chat interface. The chatbot shall:

- Recall a dermatology knowledge base of pertinent papers depending on the forecasted disease and query made by the user,
- Provide answers to passages that are not in memory (RAG),
- Describe in terms accessible to the user (not technical),
- Refuse prescription or provide incorrect dosage – suggest appointment with clinician in reasonable situations.



3.3.7 FR7 Management: Knowledge Base

Administrators will be in a position to:

- Manage documents (add, edit or remove documents (guidelines, frequently asked questions, patient manuals),)
- Retrieval from a vector database through re-indexing by trigger.

3.3.8 FR8 – Logging and Analytics

System shall record (anon: anonymization where necessary):

- The number of images that have been processed.
- Predicted classes distribution,
- Future improvement on chatbot queries (text only),
- Methods for measuring the confidence of drift detectors of models.

3.3.9 FR9 – Error Handling

- System will process poor quality of images by urging user to re-take a photo or provide a clear photo.
- System will feature easy-to-understand server error or time messages.

3.4 Non-Functional Requirements

3.4.1 NFR1 – Performance

The system should be responsive and fast in the way it interacts with the user.

- The upload of an image to the result of the classification should be between 5-10 seconds on a standard consumer device and network.
- Chatbot answers should be delivered in a few seconds to ensure a seamless conversation.

3.4.2 NFR2 – Accuracy and Reliability

The system should assure reliable secure prediction behaviour.

- Based on literature based benchmarks, the classification model should be able to achieve more than 90.38% accuracy with test datasets.
- Even in the misclassified instances the system should give conservative and safety-first recommendations.
- Always advise users to seek confirmation from health professionals.



3.4.3 NFR3 – Scalability

The system architecture will have scope for expansion and simultaneous use.

- Use horizontal scaling on the backend services.
- Use of docker containerization makes easier to deploy and scalability.
- Services should remain modular and independently deployable.

3.4.4 NFR4 – Security and Privacy

The system should be able to safeguard user data and communication paths.

- If there is communication between frontend and backend, it should be encrypted in HTTPS.
- Images uploaded: Properly store uploaded images securely with proper access control. Chat logs: Store chat logs securely with proper access control.
- Sensitive data should be secured with secure authentication means.
- Data protection principles should be adhered to (consent, purpose limitation, and minimization).

3.4.5 NFR5 – Usability

All users should be able to use the application and its operations without difficulty.

- The user interface and user experience should be mobile-friendly and responsive.
- Throughout the application, the language should be simple, and the navigability easy to understand.
- Icons, labels and tooltips should enhance the usability of the non-technical user.
- The workflow should be as simple as possible when interacting with the process.

3.4.6 NFR6 – Maintainability

The software architecture should be easily maintainable and extensible. The system needs to be modular and be separated by:

- Frontend UI
- Backend APIs
- Machine learning inference service
- RAG/chatbot service
- Database layer
- The configuration files should not be added to the code.
- Models and knowledge-base documents should not be replaced often without large changes in the code.



3.4.7 NFR7 – Ethical and Legal Requirements

System's design should follow responsible AI and healthcare principles.

- There should be a medical disclaimer on the application for the user.
- Don't be prescriptive with medical diagnosis or prescriptions.
- Users should be made aware that they should consult a medical professional as needed.
- Safety Related Info should be included in those situations that require that.

3.5 System Constraints and Assumptions

The proposed system has some technical and practical limitations. The deep learning models need to be trained using GPU resources, and run on CPU-only servers or low-resource GPUs during deployment. Bias towards skin type or location is another possible inherent problem of public dermatology datasets, impacting the generalization of models. Furthermore, the system is also internet dependent since the front end, back end, and the AI services have online API connections. Application is not intended to be a certified medical device, but rather a learning and decision making tool. The system assumes that the skin image that is uploaded is relatively clear and was taken under suitable lighting. The information in the medical knowledge base for the RAG pipeline is also assumed to be accurate and up to date.

3.6 Preliminary technology stack (Indicative)

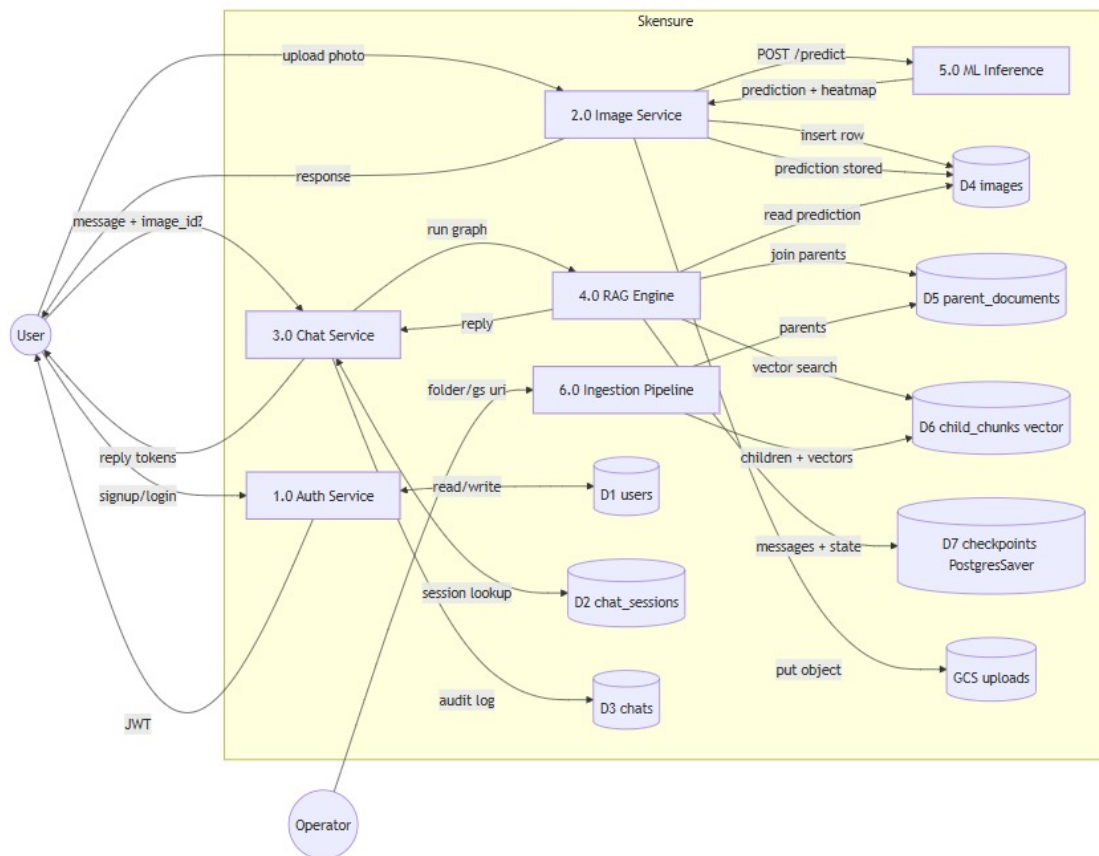
Convolutional network architecture ConvNeXt was chosen for image classification, while image preprocessing and augmentation were performed using OpenCv and TorchVision. The conversational assistant is composed of a Retrieval-Augmented Generation pipeline built on top of PostgreSQL / pgvector (for semantic retrieval), LangGraph (for orchestration) and Gemini (for response generation). The backend was created with FastAPI to run API, authentication, uploading images, and communication between services. The UI was developed with a modern approach on the frontend side with React and Vite, and for the presentation of the user interface with Tailwind CSS and Framer Motion for making it responsive and modern for both patient and doctor workflow.

3.7 System Workflow

The system's workflow starts with user authentication via the front end interface. Once the user logs in or signs up, the JWT token will be stored in the frontend and used in subsequent requests to protected APIs. Once users upload a skin image from their device gallery or camera capture, they can use the iSolve function to apply skin effects to it. After uploading the image, the back end saves the image data along with metadata and then passes on the image to the ML inference service. The ML model processes the image, applies classification with a ConvNeXt-Tiny architecture, returns prediction confidence scores, and (optionally) returns a Grad-CAM explainability heatmap for the image. The prediction result is then returned to the dashboard on the frontend where the user can see the predicted condition and confidence score. Once

analyzed, the user can still interact with the dermatology chatbot with the RAG functionality. In the chat interaction, the back-end accesses the dermatology knowledge chunks from the vector DB via similarity search, creates a contextually appropriate prompt, and submits it to the language model (Gemini) to generate a grounded response to that prompt. The response generated is returned to the user as per the role they are assigned, either patient or doctor. Predictions and conversations are persisted and made available for auditing in the PostgreSQL database.

Figure 2: Data flow for Skensure



3.8 Use Case Diagram

The use case diagram shows that the interactions between the system and its main actors are Patient, Doctor/Medical Student and System Administrator. The Patient can:

- Create account and login
- Upload skin image
- View prediction results
- Analyze the text and have a conversation with an AI assistant.
- Re-join previous chat sessions

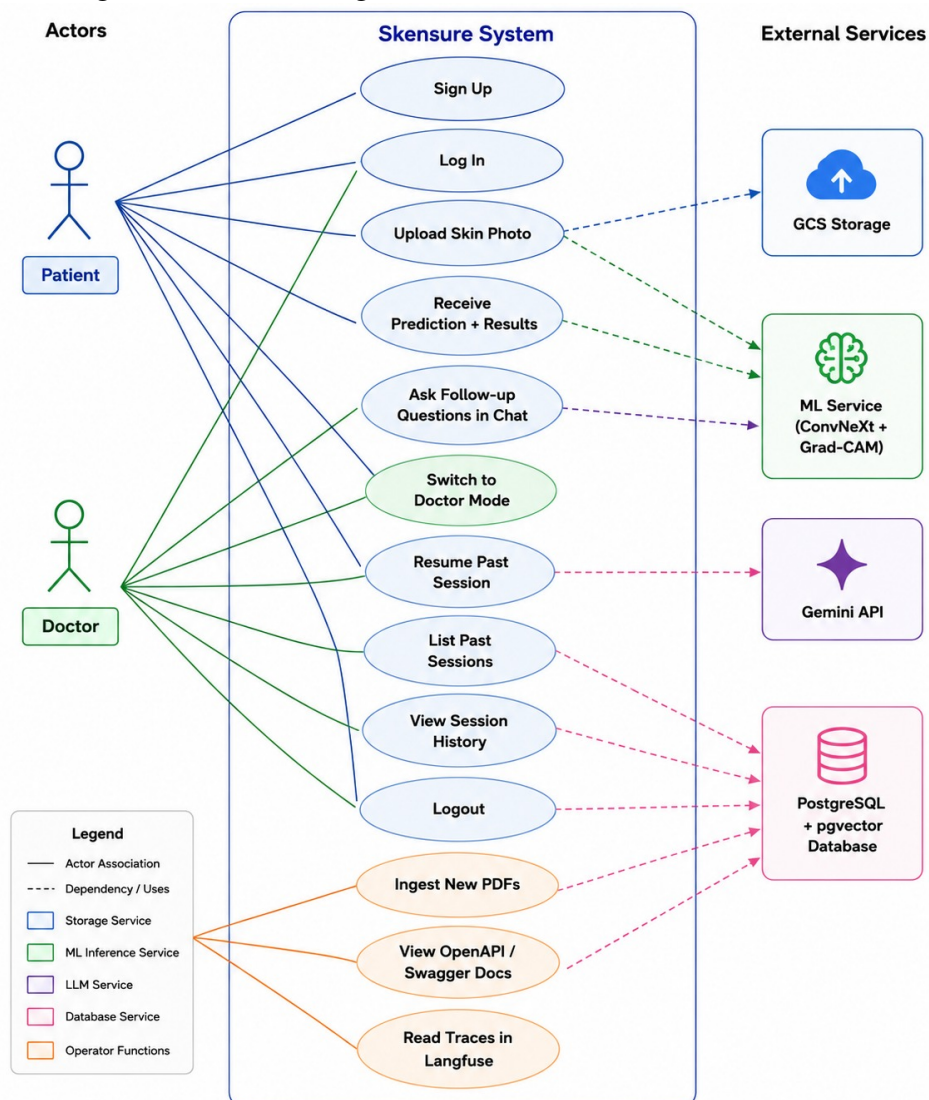
The Doctor/Medical Student may:

- Access Doctor Mode dashboard.
- View confidence scores and Grad-CAM explanations
- Check the chatbot's answers and adjust them if necessary.
- Explore differential diagnoses

The System Administrator is able to:

- Manage documents in knowledge base. Manage knowledge base documents.
- Automatically ingest and re-index PDFs:
- Ensure logs and system performance are monitored

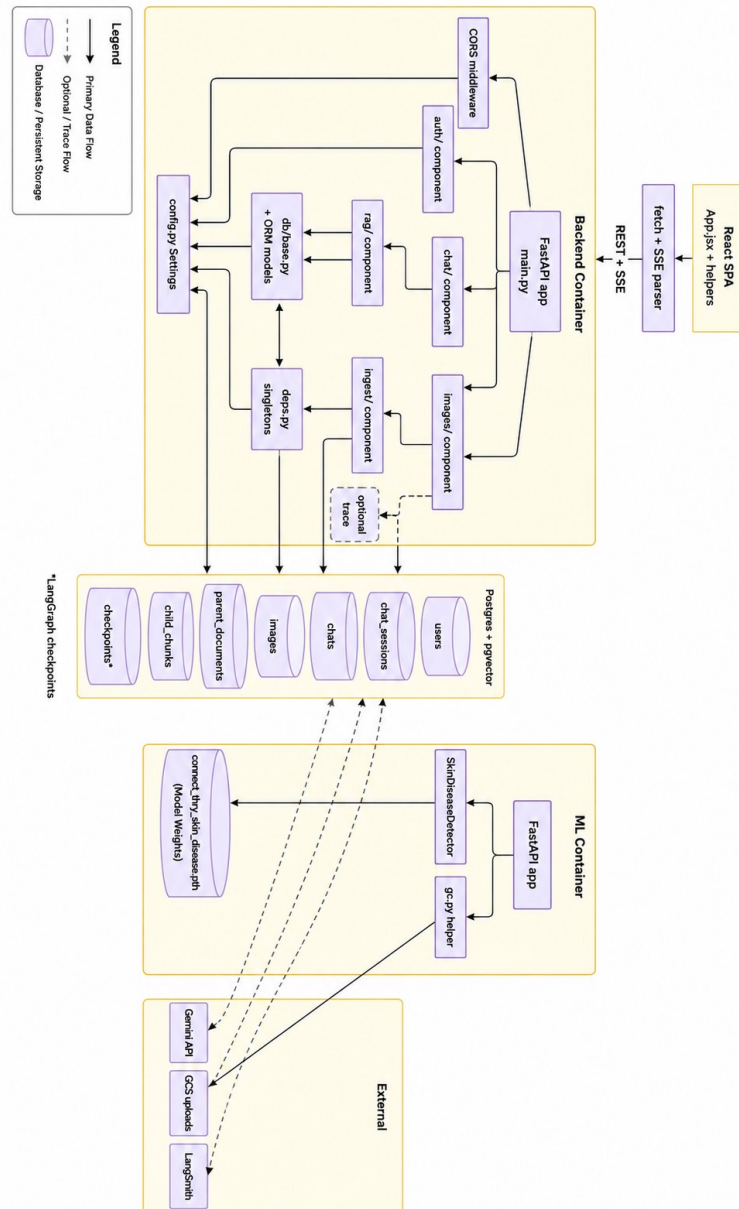
Figure 3: Use case diagram for Skensure actors and interactions



3.9 System Architecture

The proposed system is built on a modular 3 service architecture that includes a React frontend, a FastAPI backend, and a standalone ML inference service. Frontend is used for authentication, image upload, prediction visualization, and chatting with the chatbot. The backend is responsible for session management, authentication, communication between services, persistence of chat, and retrieval of the vectors. The remaining task of the ML service is to only preprocess the image, classify the object in the image, and generate Grad-CAM. Semantic retrieval is implemented via the RAG subsystem that is based on LangGraph orchestration, PostgreSQL, and pgvector. Vector searchable chunks are created from the ingestion pipeline of dermatology guideline PDFs. Chunks retrieved and sent to the Gemini language model for grounded responses. Containerization and service management with Docker Compose for local reproducible deployment and easy cloud deployment.

Figure 4: High-level architecture of the system



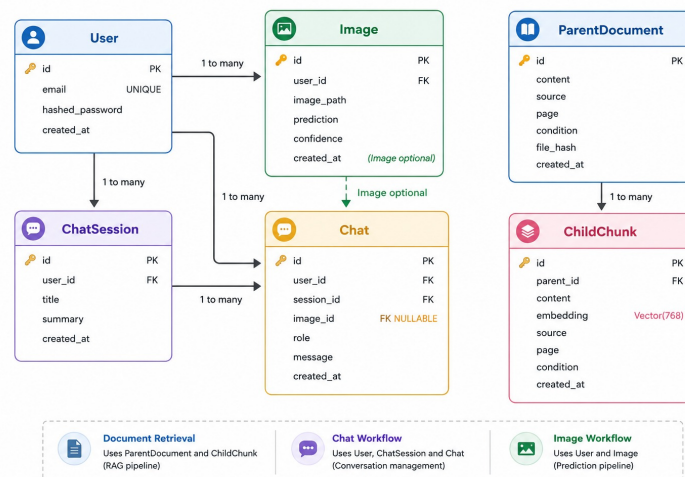
3.10 Database Design / ERD

The system database is based on PostgreSQL (which is extended with pgvector). The database holds vector embeddings for semantic retrieval along with relational application data. The key players are:

- Users
 - Saves the user account information and roles
- Images
 - Uploads metadata and prediction results for images.
- Chats
 - Saves user conversation log
- Chat Sessions
 - Records each conversation session
- Parent Documents
 - Saves original PDF chunks of content
- Child Chunks
 - Represents and accesses semantic chunks in a vector format
- Graph Checkpoints
 - Persists the conversation state of langGraph

Relationships between entities guarantee that images uploaded, predictions, and chatbot conversations are associated with authenticated users, and secure filtering of ownership. The ERD also enables retrieval of RAG by linking child vector chunks to parent medical documents.

Figure 5: Entity-relationship-diagram for RAG





3.11 Summary

The functional and non-functional requirements, user roles, user workflows, architecture, and database design of the system were defined in this chapter. The analysis laid the technical groundwork by outlining the technologies to consider, constraints of the system, and the various modules needed. The implementation phase that is discussed in the next chapter followed these specifications.

Chapter 4

System Design and Implementation

4.1 Introduction

This chapter highlights the technical design, implementation approach, and architectural decisions made during the creation of the project. The system consists of a reactive front end created with React, high-performing FastAPI back-end, a classification pipeline based on deep learning, and RAG-based chatbot controlled by LangGraph.

4.2 Overall System Workflow

In a nutshell, the process goes like this: the user uploads a photo of the skin on the React front-end; the image is passed on to the FastAPI backend as part of the multipart request; the back-end pre-processes the image with the OpenCV preprocessing pipeline and then passes it to the ConvNeXt-Tiny model for inference; the output values consisting of the prediction, confidences of the classes, and Grad-CAM are then returned back to the front-end; finally, the user interacts with the LangGraph-controlled chatbot, which accesses relevant information from the pgvector database to generate a proper reply. [Figure 4]

4.3 Frontend Design and User Interface

The user interface has been developed using React 18, with Vite providing fast builds and optimized bundling of code and assets. The front end uses Tailwind CSS for responsive layouts and animations done by means of Framer Motion without blocking the main rendering thread.

In terms of the user interface, role-based design was used. When authenticating themselves, users can pick between two modes – Patient and Doctor/Medical Student. This choice influences the components rendered in the application:

- Patient mode: complex citation removed, the user is provided with the disclaiming message;
- Doctor mode: advanced options like Grad-CAM heatmaps are available along with medical source citations within the chat.

Images are uploaded by means of a custom-made dual-mode input component. Users have the option to either upload previously stored pictures in jpeg/png formats, or take a new picture using their phone camera (via `navigator.mediaDevices.getUserMedia`). Images are taken in a 720p quality and are transferred into the off-screen canvas object before being converted to JPEG format.



4.4 Backend Architecture and API Integration

The backend system was built using FastAPI (Python 3.11+) due to its ability to process requests asynchronously and generate auto-generated OpenAPI documentation. The application followed the modular and decoupled architecture below:

- `main.py` file was in charge of endpoint routing and dependency injection.
- `database.py` and `models.py` files contained the settings for the SQLAlchemy ORM configurations.
- `langgraph.py` file contained the code responsible for the state machine implementation during LLM conversations.
- `model.py` file served as the core wrapper module for loading and executing PyTorch vision models.

Conversational memory was maintained in a `ChatState` typed dictionary, containing the current `session_id`, `user_id`, recently fetched vector chunks, and `image_id`.

Communication between client and server was carried out using RESTful endpoints:

- **Auth Routes:** `/signup`, `/login` routes authenticated the provided credentials and returned metadata, including user access roles.
- **Image Processing:** `/upload-image`, `/predict/{image_id}` routes handled the image processing pipeline. Upload endpoint expected `multipart/form-data` content-type, stored the binary file under a generated uuid name, and recorded the file path into PostgreSQL. Predict endpoint invoked the lazy-loaded PyTorch model.
- **Chat Orchestration:** `/chat` and `/chat-session` accepted the JSON payload containing user input and current active image context.

4.5 AI Model Development

4.5.1 Dataset Description

The core of the model is very carefully selected, clinically relevant data set, with different resolution dermatological images. The data set is completely organized into five separate diagnostic classes to include common and highly complex communicable skin conditions that are:

- Acne Rosacea
- Atopic Dermatitis
- healthy
- Chickenpox
- Shingles



To guarantee statistically valid model training and testing, the data was randomly divided into 70% training set, 15% validation set, and 15% test set using a fixed random number generator seed to ensure exactness of the reproduction. In order to overcome the serious class imbalance, where there are only a few medical data, an offline augmentation approach was used. Only used on the training split, and oversampling each class until a total of 2,000 images per class examples were reached. The validation and test splits were left completely unaugmented to prevent leakage of data and unbiased evaluation. The use of a “Healthy” control group is the key to its real world deployment, so that the model can accurately detect and does not try to make a pathological diagnosis where there is no disease present.

4.5.2 Data Preprocessing Techniques

The lighting level, skin color, orientation of the image and camera characteristics of dermatological images may vary. All images were preprocessed using a standard pipeline to ensure that the features extracted are comparable, and the backbone used for pretraining is ConvNeXt-Tiny.

1. **RGB Standardization:** All images were converted to RGB to ensure consistency of images collected from different sources and different types of devices.
2. **Image Resizing:** The images were resized to a fixed size of 224×224 , which is the input size required by the pretrained ConvNeXt-Tiny architecture.
3. **Tensor Conversion:** The resized images were converted to PyTorch floating point tensors for deep learning operations.
4. **Image Normalization:** Pixel values were normalized using the standard ImageNet statistics:
 - Mean: [0.485, 0.456, 0.406]
 - Standard Deviation: [0.229, 0.224, 0.225]

In addition to a simple normalization, a straightforward approach to aligning the distribution of the input data with the ImageNet data set on which ConvNeXt-Tiny was originally pretrained.

4.5.3 Data Augmentation Techniques

Large amounts of data are a prerequisite for the effective generalization of deep learning models. The pipeline has a very effective two-stage augmentation strategy:

1. **Offline Augmentation (Dataset Balancing):** The training split is performed offline, before training. The number of aggressively supplemented was increased by a stand-alone script to the goal of 2000 images per class. The following changes were made:
 - **Random Horizontal Flip ($p = 0.5$):** Simulates natural horizontal variation, but maintains disease characteristics.
 - **Random Vertical Flip ($p = 0.5$):** makes the images more oriented and increases the invariance to image positioning.



- **Random rotation ($\pm 20^\circ$):** Provides realistic viewpoint variations without making significant changes to the structures of lesions.
- **Color Jitter:** Applies a random change to brightness, contrast, saturation and hue to produce a variety of lighting and imaging situations.
- **Random Affine Translation (10%)** Simulates minor camera motion and misalignment of images.

This offline augmentation process not only balanced the classes, but also also enhanced the effective training size without losing clinically relevant visual features.

2. **Online Augmentation (Dynamic Regularization):** This technique adds noise to the training set in the training loop. It applies mild augmentation to the already enlarged data set on-the-fly. The online augmentation pipeline consisted of:

- **Random Resized Crop (Scale 0.8–1.0):** Promotes models to be aware of the areas of lesions whilst preserving vital diagnostic data.
- **Random Horizontal Flip ($p = 0.5$):** Makes the model more robust to changes in orientation from left to right.
- **Random rotation ($\pm 20^\circ$):** Mildly adds rotation in training.
- **Color Jitter:** Simulates Illumination, contrast and color shifts.
- **Random Affine Translation (10%):** Makes the model more robust to small random translations.
- **MixUp Augmentation ($\alpha = 0.2$):** Generates blended training samples and soft labels for better generalization and to avoid overconfidence.

The offline and online augmentation allowed ConvNeXt-Tiny to learn strong representations of the diseases and reduced overfitting effects, enhancing the performance on unseen images.

4.5.4 Evaluated Deep Learning Models

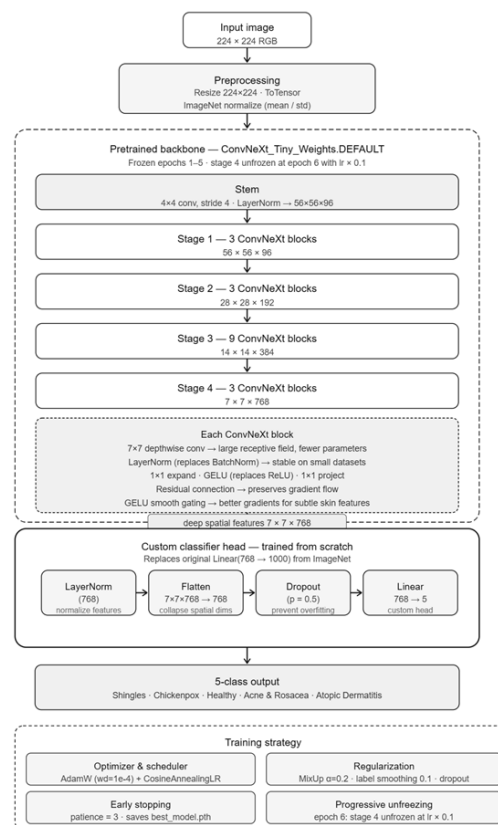
For optimal accuracy, two very different computer vision paradigms were evaluated:

1. **Pretrained ResNet50 (Hybrid/Two-Stage Approach):** A Residual Network (ResNet50), deep feature extractor has been used. The final fully connected classification head was removed. The high dimension feature vectors obtained by the convolutional base were then fed into a Support Vector Machine (SVM) using a Radial Basis Function (RBF) kernel.
2. **Pretrained ConvNeXt-Tiny (End-to-End Deep Learning):** A modern, highly optimized Convolutional Neural Network trained entirely end-to-end, serving as the primary candidate for the final deployment.

4.5.5 ConvNeXt Architecture

The ConvNeXt-Tiny architecture was adopted as the final model as it is a modern and effective visual recognition architecture. In order to save the time of training the model from scratch, the pre-trained backbones from ImageNet were used as initial weights. The classification head was modified from 1000 classes of the ImageNet to the five diagnostic classes. In the custom classifier head, a Dropout layer with 0.5 dropout rate was inserted before the final Linear layer, and then connected to Linear(768, 5) that maps the extracted deep spatial features to the 5 classes of skin diseases. In the training stage, the feature extractor was frozen to allow only the classifier head to learn task specific patterns. Later, the last feature stage was unfrozen to tune deeper features for improved classification performance. This architecture is shown in Figure 6.

Figure 6: ConvNeXt Architecture



The main architecture benefits of ConvNeXt are:

1. **Efficient Feature Extraction:** ConvNeXt-Tiny provides strong feature extraction while remaining lightweight enough for faster training and deployment.
2. **Large Kernel Sizes:** employs 7 x 7 convolutional kernels, which is much larger.
3. **Contextual Learning:** network can then learn the meaning of the skin and its context so that it can understand the bigger picture of a skin lesion.



4. **Modern Activations and Normalization:** learnable modern activations replace standard ReLU and Batch Normalization with GELU and Layer Normalization, resulting in much smoother gradient flow.

4.5.6 Model Training Configuration

The hyperparameters were strictly controlled to ensure reproducible training cycles, as summarized in Table 2.

Table 2: Training Hyperparameters and Justifications

Hyperparameter	Value	Justification
Epochs	10	Provides enough training cycles for the classifier head and fine-tuning stage while avoiding unnecessary overtraining.
Batch Size	32	Balances GPU memory usage, gradient stability, and convergence speed.
Optimizer	AdamW	Decouples weight decay from gradient updates, improving regularization and generalization during transfer learning.
Initial Learning Rate	0.0001	A conservative learning rate suitable for fine-tuning a pretrained ConvNeXt-Tiny model without causing unstable updates.
Backbone Fine-Tuning Learning Rate	0.00001	The final backbone stage was fine-tuned using a learning rate 10 times smaller than the initial rate to preserve pretrained ImageNet features.
Weight Decay	1×10^{-4}	Applies mild regularization to control model complexity without causing underfitting.
Loss Function	Cross-Entropy Loss with Label Smoothing (0.1)	Suitable for multi-class classification while reducing overconfident predictions.
Dropout Rate	0.5	Reduces overfitting by preventing excessive dependency on specific classifier neurons.
Learning Rate Scheduler	CosineAnnealingLR	Gradually reduces the learning rate during training, allowing smoother convergence and improved final optimization.
Progressive Fine-Tuning	Backbone frozen for epochs 1–5; final stage unfrozen from epoch 6	Allows the custom classifier head to learn first before adapting deeper ConvNeXt features to dermatology images.
Early Stopping Patience	3	Stops training after three non-improving validation epochs to reduce overfitting and save the best model checkpoint.



4.5.7 Performance Optimization Techniques

The ConvNeXt-Tiny model was optimized and regularized using various methods to enhance its stability and performance. The model was trained with ImageNet-pretrained weights and progressively fine-tuned by freezing the backbone for the first five epochs and then un-freezing the backbone from epoch six with 0.1 times of the learning rate. Cosine annealing was used with AdamW optimizer to achieve smoother convergence with learning rate= 0.0001 and weight decay= $1e-4$. Dropout(0.5), label smoothing 0.1, MixUp 0.2, and early stopping 3 were used to reduce overfitting and the best model was saved as *best_{model}.pth*.

4.5.8 Final Model Selection

After a thorough empirical testing process, ConvNeXt-Tiny was chosen as the final production-ready model. It significantly outperformed the ResNet50-SVM hybrid model, achieving an excellent 92.31% test accuracy. The hybrid model achieved an average accuracy of 90.38%, compared to ConvNeXt-Tiny's 92.31% accuracy on strictly unseen data. The clarity of the end-to-end architecture allowed the loss function to be directly backpropagated and optimized through ConvNeXt. The convolutional layers were specifically adapted for the five target classes, and the final model weights were serialized and saved for deployment.

4.5.9 Image Processing and Prediction Pipeline

A full-stack prediction pipeline was engineered and optimized for deployment to end users:

1. **Data Ingestion:** The user uploads a skin image through the React frontend or captures a live image using the built-in browser camera interface. The image is sent as a `multipart/form-data` HTTP request to the FastAPI backend.
2. **Backend Processing:** The FastAPI backend validates the uploaded file, authenticates the user, saves image metadata in the PostgreSQL database, and passes the image to the dedicated machine learning inference service.
3. **Inference Execution:** The machine learning service preprocesses the image by resizing and normalizing it according to the ConvNeXt model requirements. The resulting tensor is processed by the trained ConvNeXt model, and the model output is passed through a Softmax activation function to obtain probability confidence scores.
4. **Explainable AI (Grad-CAM) Generation:** The pipeline employs Gradient-weighted Class Activation Mapping (Grad-CAM) for explainable AI to support clinical transparency. The system generates gradients of the predicted class by backpropagating through the last 7x7 convolutional block of the ConvNeXt model. This creates a coarse localization map indicating the regions of the image that contributed most to the diagnosis. The map is then upsampled, converted into a heatmap, and overlaid on the original image.
5. **Response Delivery:** The machine learning inference service returns the predicted skin condition, confidence scores, probability distribution, and Grad-CAM heatmap visualization to the FastAPI backend in JSON format. The prediction results are stored in the backend and passed to the React frontend, where they are dynamically displayed in the interface.



4.6 RAG-Based Medical Chatbot

To overcome common pitfalls of standalone generative AI systems, such as hallucinations and poor grounding, the chatbot was developed using a Retrieval-Augmented Generation (RAG) framework. This improves response grounding by allowing the chatbot to access a pre-curated dermatology medical document corpus instead of relying only on the language model’s internal parametric memory. The system integrates image prediction, context-based conversation memory, semantic vector retrieval, metadata filtering, multilingual support, and large language model generation into a single conversational flow.

The RAG pipeline starts when the user creates a message or adds a dermatology image. When an image is provided, the ConvNeXt image classification service predicts the most likely condition and returns a heatmap to identify important image regions. Next, the chatbot detects the language and condition context of the text to identify the active conversational condition. The user query is then rewritten into an optimized English retrieval query for searching the English medical corpus.

The system is implemented with pgvector cosine similarity search over embedded child chunks in PostgreSQL. The retriever oversamples candidate chunks, filters them using metadata and condition-aware gating, removes duplicate parent documents, and returns the highest-similarity results. For comparison queries, such as “rosacea vs acne,” the system performs multi-condition retrieval fan-out to ensure that evidence for both conditions is retrieved. The retrieved chunks are then assembled into a strict system prompt that instructs the LLM to answer only from the provided knowledge and avoid unsupported medical recommendations or diagnoses.

The chatbot also includes a language-aware orchestration mechanism that supports Urdu-script queries. For semantic search, Urdu queries are converted into English retrieval queries, while the final responses are returned in Urdu script. This enables multilingual interaction without requiring a separate Urdu vector database or Urdu medical corpus.

A large instruction-following model is adopted for grounded response generation in the language model layer. The system dynamically builds prompts using retrieved medical chunks, conversation history, image prediction summaries, and safety constraints. Streaming responses are supported through Server-Sent Events (SSE), improving interactivity by allowing token-level response generation.

Medical PDFs were ingested using a parent-child chunking strategy. RecursiveCharacterTextSplitter divided documents into large parent chunks of 1500 characters with 200-character overlap, which were further split into smaller child chunks of 400 characters with 80-character overlap. Child chunks were embedded into 384-dimensional vectors using sentence-transformers and stored in the pgvector child_chunks table. Batched ingestion with batch_size=32 prevented RAM exhaustion during embedding.

4.6.1 Database Integration

Database integration was implemented using SQLAlchemy ORM, PostgreSQL, and pgvector. SQLAlchemy provides an abstraction layer for database models, relationships, queries, and



transaction management, while PostgreSQL acts as the centralized storage engine for conversational, authentication, image, and retrieval data. The backend accesses the database through FastAPI dependency injection with session-scoped database connections.

The overall architecture combines relational storage and vector search. User information, chat history, image predictions, and conversational sessions are stored as structured relational data, while semantic embeddings generated during ingestion are stored as vector fields through pgvector. This design allows structured SQL filtering and semantic similarity search to operate together within the same database system.

The retrieval subsystem uses parameterized SQL queries with pgvector cosine similarity search. The user query is represented as an embedding vector and compared with stored child-chunk embeddings using the $\leq$ cosine distance operator. Retrieval quality is improved through oversampling and similarity filtering, reducing irrelevant retrieved items. As the medical corpus grows, HNSW indexing can significantly improve vector retrieval speed and enable low-latency semantic search across a large number of documents.

Additionally, LangGraph integration persists graph checkpoints in PostgreSQL through the PostgresSaver mechanism. Checkpoints, graph state, and conversational continuity are stored in dedicated checkpoint tables, enabling session continuity and recovery across conversational turns. The graph singleton and database connection pool are initialized when the application starts to avoid repeated initialization overhead and improve runtime performance.

4.6.2 Authentication and Security

The project supports Bearer JWT authentication and bcrypt password hashing to secure access to application resources. During sign-up, user passwords are hashed with bcrypt using randomly generated salts before being stored in the PostgreSQL database. The API does not return, log, or store plaintext passwords. JWT tokens are signed with the HS256 algorithm and include claims for user identity, role, issue time, and expiration time. Authenticated requests include the token in the `Authorization: Bearer` header.

The authentication flow includes sign-up, login, token validation, and ownership-based authorization checks. Error messages for incorrect usernames and passwords are intentionally kept generic to discourage user enumeration attacks. Protected endpoints validate tokens and check whether the associated database user exists.

Authorization is based on ownership, where each protected resource is linked to a specific user. Conversations, uploaded images, and chat history are filtered by `user_id` to prevent unauthorized access to another user's resources. Requests for foreign sessions return the same 404 response pattern and do not reveal whether the session exists.

Several additional security enhancements were included in the backend design. SQLAlchemy ORM queries and parameterized SQL statements reduce SQL injection risks. React's default HTML escaping mitigates Cross-Site Scripting (XSS) risks. The use of Bearer tokens instead of browser cookies helps reduce Cross-Site Request Forgery (CSRF) risks. File upload validation prevents empty uploads and unsupported file types. In production, communication



channels between the browser, backend, cloud storage, and external AI services are accessed through HTTPS.

The system also includes prompt-level protection against common LLM vulnerabilities such as prompt injection and hallucination. The system prompt instructs the language model to use only the provided medical knowledge and avoid unsupported diagnosis or prescription behavior. Retrieval-gated prompting and similarity thresholds further reduce hallucination risk by tying responses to retrieved medical sources.

4.6.3 System Interfaces and Screens

The project interface presents users with the following key screens:

- **Login/Registration Screen:** Supports account creation and role selection for patients and doctors.
- **Image Upload Screen:** Provides dual-mode input through file upload and live camera capture.
- **Prediction Result Screen:** Displays the predicted skin condition, confidence score, probability distribution, and Grad-CAM heatmap visualization.
- **Medical Chatbot Screen:** Allows users to ask condition-related questions and receive grounded RAG-based medical responses.
- **Conversation History Screen:** Stores and displays previous user conversations and prediction sessions for continuity.

4.7 System Deployment

Proposed system deployment architecture uses three services frontend, backend and machine learning inference. It is an architecture where the user interface, application logic, and AI inference are broken down into independently deployable services and are developed, scaled and maintained independently. The deployment configuration is designed to enable both a local Docker Compose environment for development and a cloud-based production environment with the following Google Cloud Platform services: Cloud Run, Cloud SQL, Google Cloud Storage, Firebase Hosting and Secret Manager.

4.7.1 Target Deployment Environments

There are two deployments options. The first environment is for local development and testing purposes, and is meant to run the full stack using Docker Compose on a single machine. The front-end, back-end, machine learning service, PostgreSQL database, image storage for uploaded images and a document ingestion pipeline are all contained in a single local environment.

The second environment is the production deploy where Google Cloud Platform is used. Both the frontend as well as the backend and machine learning service are deployed independently



via Cloud Run and Firebase Hosting, respectively. Cloud SQL provides support for PostgreSQL with pgvector support, Google Cloud Storage provides for storing uploaded images, and Secret Manager for the management of sensitive credentials. This separation allows for growing the number of entities as services and will help to keep service responsibilities clear.

Table 3: Supported Deployment Environments

Environment	Purpose
Local (Docker Compose)	Development and testing on a single machine
Production (Google Cloud Platform)	Scalable cloud deployment using managed infrastructure

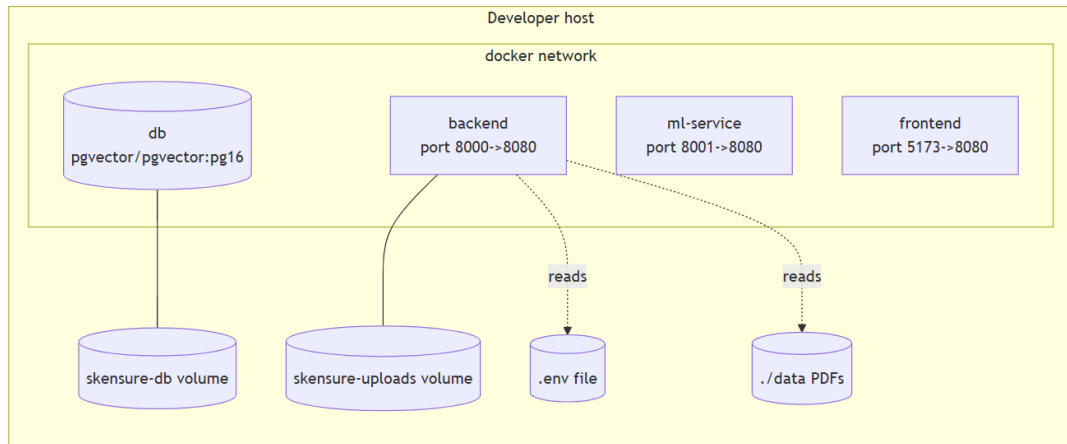
4.7.2 Local Deployment Using Docker Compose

Four main containers are used to orchestrate the environment that is being deployed locally: PostgreSQL database container, the backend container(s), the machine learning inference container, and the frontend container. These containers run in the same Docker network, so that services in the Docker network can communicate with each other without needing to specify any extra networking parameters. The pgvector-enabled image is used by the PostgreSQL container to enable semantic retrieval operations that are used in the Retrieval-Augmented Generation pipeline. Docker volumes are persistent and store the data in the database along with the uploaded image, which will survive container restarts. The backend also mounts the dermatology PDF corpus at `./data` for ingesting and generating embedding of documents.

The frontend is running on `localhost:5173`, the backend service is running on `localhost:8000` and the machine learning inference service is running on `localhost:8001`. Starting up the sequence using dependency ordering and health checks. Beginning initialization of the backend service, the container needs to be healthy, which is checked by the `pg_isready` health check. The backend then runs Alembic database migrations and starts the Uvicorn-based FastAPI application.

The dermatology knowledge corpus is ingested after the first deployment, by commanding as follows: `docker compose exec backend python -m app.ingest /srv/data` It breaks the dermatology PDF files, creates embeddings for them and stores them within PostgreSQL, which is enabled to embed them using pgvector for semantic retrieval operations later.

Figure 7: Local Deployment Using Docker Compose



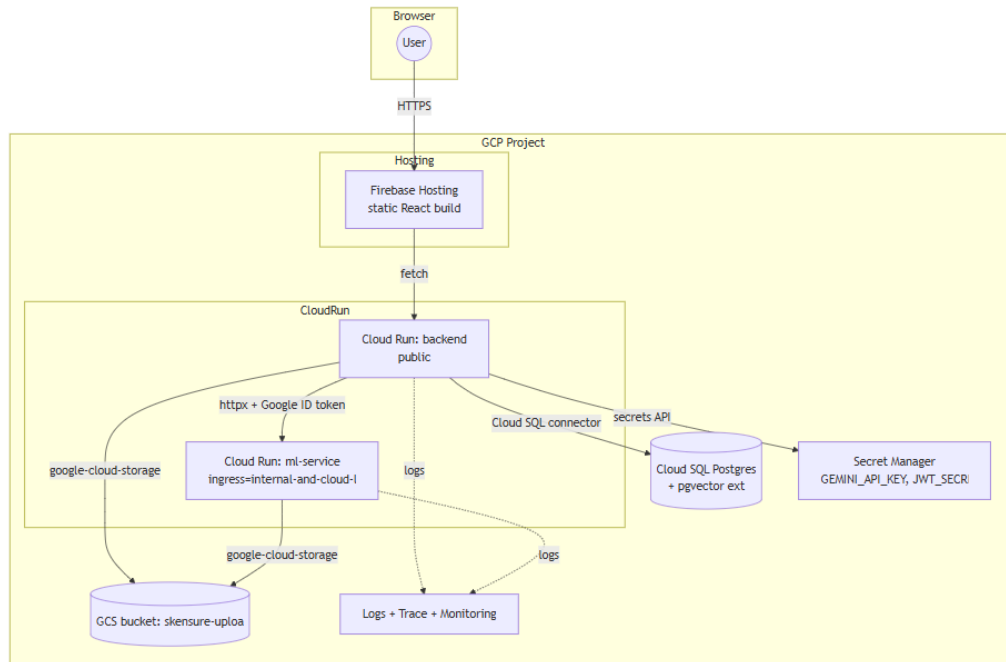
4.7.3 Production Deployment on Google Cloud Platform

The frontend is deployed on its own from the backend services in the production environment. React front end is optimized and bundled as static files with Vite and hosted on Firebase Hosting; served via Google's CDN system with HTTPS.

The backend service is deployed to Cloud Run as a public service for authentication, handling of image uploads, orchestration of the chatbot, communication with the database, and communicating with the machine learning inference service. Images are stored in Google Cloud Storage and persistent records and vector embeddings are stored and managed in Cloud SQL with pgvector support.

Machine learning service is deployed independently to Cloud Run and the ingress is restricted by internal and cloud load balancing. This will block public access to the inference endpoint. Rather, the backend calls the ML service in isolation and with Google issued ID tokens sent upon outbound calls. Secret Manager can provide secrets like the Gemini API key, the JWT signing secret to store sensitive values without embedding them directly in the source code or container images.

Figure 8: Production Deployment Architecture on GCP



4.7.4 IAM Roles and Service Permissions

The deployment uses IAM with dedicated service accounts for the backend, and machine learning operations according to a least-privilege approach. The backend service account obtains access to the private ML service, to cloud SQL, to the upload bucket and to secrets in Secret Manager. During inference operations the machine learning service account is given read only access to images uploaded for the operation.

Table 4: IAM roles and service permissions.

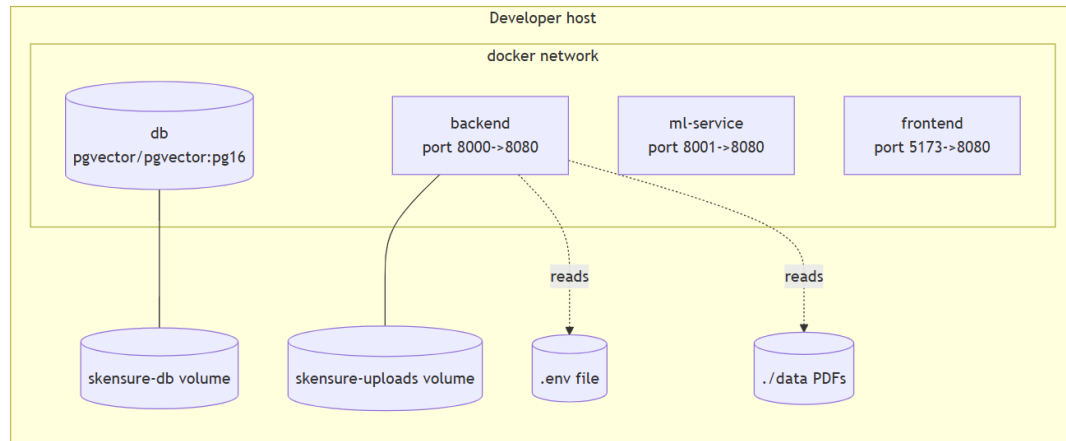
Service Account	Role	Purpose
Backend Service Account	Cloud Run Invoker	Invoke private ML service
Backend Service Account	Cloud SQL Client	Database communication
Backend Service Account	Storage Object Admin	Upload and retrieve images
Backend Service Account	Secret Manager Accessor	Retrieve API secrets
ML Service Account	Storage Object Viewer	Access uploaded images

4.7.5 Environment Configuration

The environment is set by Environment Variables to provide flexibility in deployment Local and Cloud way. Backend configuration variables are used to set up the connection to the database, JWT configuration, Gemini API credentials, embedding model settings, ML service URL, Google Cloud Storage bucket configuration and CORS origins. The machine learning

service configuration contains the model weight path for the ConvNeXt model, and inference configuration. This is a configuration based on the environment that can be utilized to move services from one deployment target to another without editing the application code.

Figure 9: Production Deployment Architecture on GCP



4.7.6 Scaling and Cold-Start Strategy

In the deployment architecture, different services have different strategies for their scaling. There are at least one instance of the back-end service running, and up to 5 instances when there is more traffic. This cuts down on delays from reconnecting to third-party AI tools like Gemini. The machine learning inference service has a minimum of 0 and a maximum of 3 instances. The model weights are about 5 - 8 seconds to scale in and are fine to scale in to zero during idle time, while also scaling out during increased demand.

Table 5: Service scaling configuration.

Service	Minimum Instances	Maximum Instances
Backend Service	1	5
ML Service	0	3
Firebase Hosting	Managed Automatically	CDN-based

4.7.7 Networking and Security

The production deployment is via HTTPS and handled by TLS on Cloud Run. Restricted ingress policies are used to protect the inference service in the ML program, thus keeping it inaccessible to public clients. Backend-to-ML communication is protected by Google issued ID token with internal requests. CORS policies grant access to the origin of the web app that is hosting the frontend. The deployment also includes optional VPC connectors for traffic to Cloud SQL to go through private IP networking. Sensitive configuration values are retrieved dynamically at runtime from Secret Manager, and are not stored in Docker images.



4.7.8 Backup, Recovery, and Observability

Important system assets are backed-up and monitored in the deployment strategy. Automated Cloud SQL daily backups with point-in-time recovery provides protection of PostgreSQL records. Use bucket versioning to add another level of security to uploaded images in Google cloud storage. The cloud logs are sent to Cloud Logging and the cloud execution and interaction with Gemini are traced using LangSmith. Request latency, number of instances and error rates are measured for both services and measured using uptime checks on the /healthz endpoints of both services.

4.7.9 Build and Rollout Process

When the version tag is pushed to the repository, the rollout process starts. The development of Docker images for the backend and machine learning service, and then pushing them to Google Artifact Registry. Those migrations are then performed via the Cloud SQL Proxy and the services are then deployed to Cloud Run using gcloud run deploy. The frontend is deployed separately with the help of firebase deployment commands. This deployment process enables independent deployment of the frontend, backend and the ML service while ensuring controlled updates for the production environment.

4.7.10 Suitability of Cloud Run

Cloud Run was chosen due to its container support capabilities, automatic scaling options and IAM integration support, as well as its scale-to-zero characteristics. While offering the flexibility of running microservices, Cloud Run also offers an operational model that's simpler than other options like Kubernetes Engine, App Engine or Compute Engine. This will minimize the management overhead of infrastructure, and preserve deployment flexibility and scalability of the planned system.

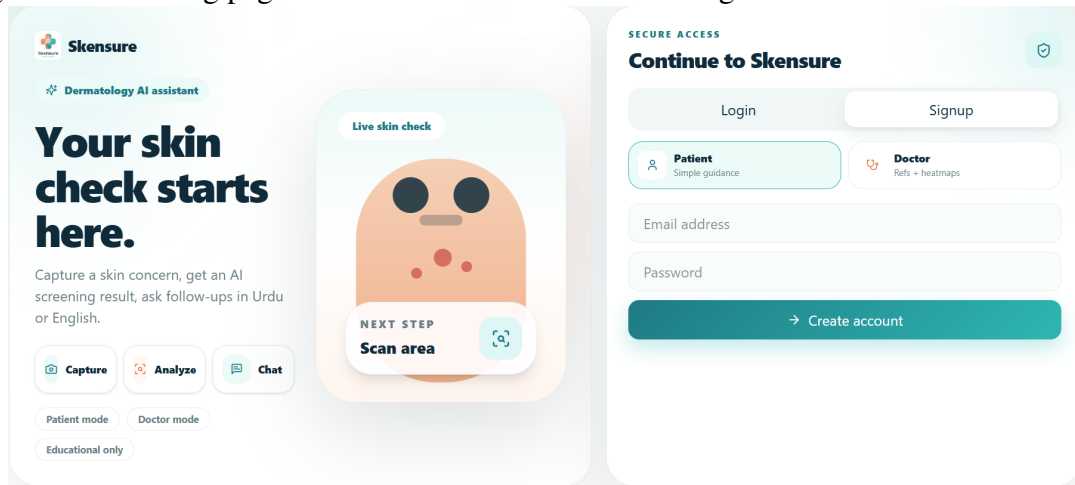
4.8 System Interfaces and Screens

The user interface can be used in two different modes: in a patient centric mode, emphasizing accessibility and communication in plain language; and in a doctor or medical student mode, presenting more clinical information and explanation output. They both have the same technological foundations but vary in terms of the set of words that can be responded to by the chatbot, the amount of analysis after each scan and the follow-up questions that are given to the user.

4.8.1 Landing Page and Authentication

The entry point of the application offers a brief value proposition as well as a role selector (patient, doctor, educational); providing a user with some context regarding the capabilities of the system before he logs in. The login/signup form is safe and secure to use to handle authentication. Once user logs in this application detects and remembers the selected mode and directs the user to an appropriate configured dashboard session.

Figure 10: Landing page and authentication screen showing role selection and secure login



4.8.2 Image Upload and Analysis Interface

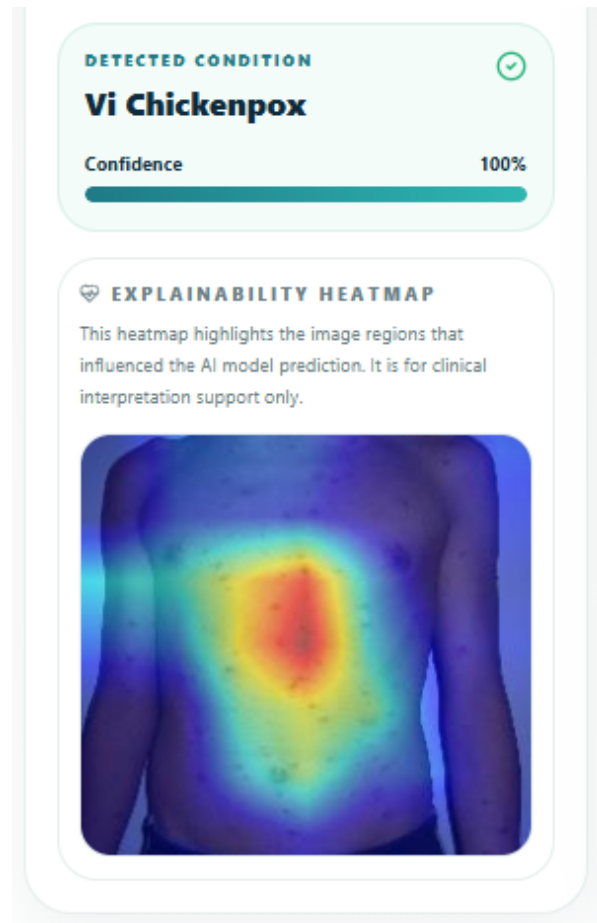
The main dashboard is an application's backbone that runs the functional business. A skin scan panel is a panel that is able to receive input in the form of an image, which can be uploaded from a file, or captured by a device camera, and can be in PNG, JPG or JPEG format. After choosing an image, the user will see a preview of the image in the Panel before starting to analyze it. When the image is uploaded, it sends the image to the inference endpoint of the FastAPI app, which then runs the ConvNeXt classification model on the image.

Figure 11: Skin image uploaded and ready for analysis



The classification result comes back with a detected condition label and the percentage of confidence. When in doctor mode, this is also accompanied by a Grad-CAM heatmap overlaid to the original image, which shows the areas that most influenced the model when predicting the image. This explainability feature is to aid in the clinical interpretation and is only shown in the clinical access mode, recognising that this technical output is dependent upon domain expertise to accurately interpret.

Figure 12: Detected condition with confidence score and Grad-CAM explainability heatmap (Doctor Mode)



4.8.3 Doctor Mode Dashboard

If the user logs in in doctor or medical student mode, the doctor or medical student mode will be displayed on the header on the dashboard. In this mode, the chatbot panel can be used as a clinical reference assistant, with suggested prompts following a scan, such as 'Explain the clinical significance of the result', 'Investigate differential diagnoses for the scan' or 'Discuss the management implications of the scan'. In this mode chatbot answers reference sources from which information has been taken, giving the reader a clear reference to the source literature. This is the citation behaviour that is directly caused by the RAG pipeline that adds provenance information to every instance of retrieved text before feeding it into the language model.

Figure 13: Doctor mode dashboard showing the skin scan panel and dermatology assistant with clinical prompts

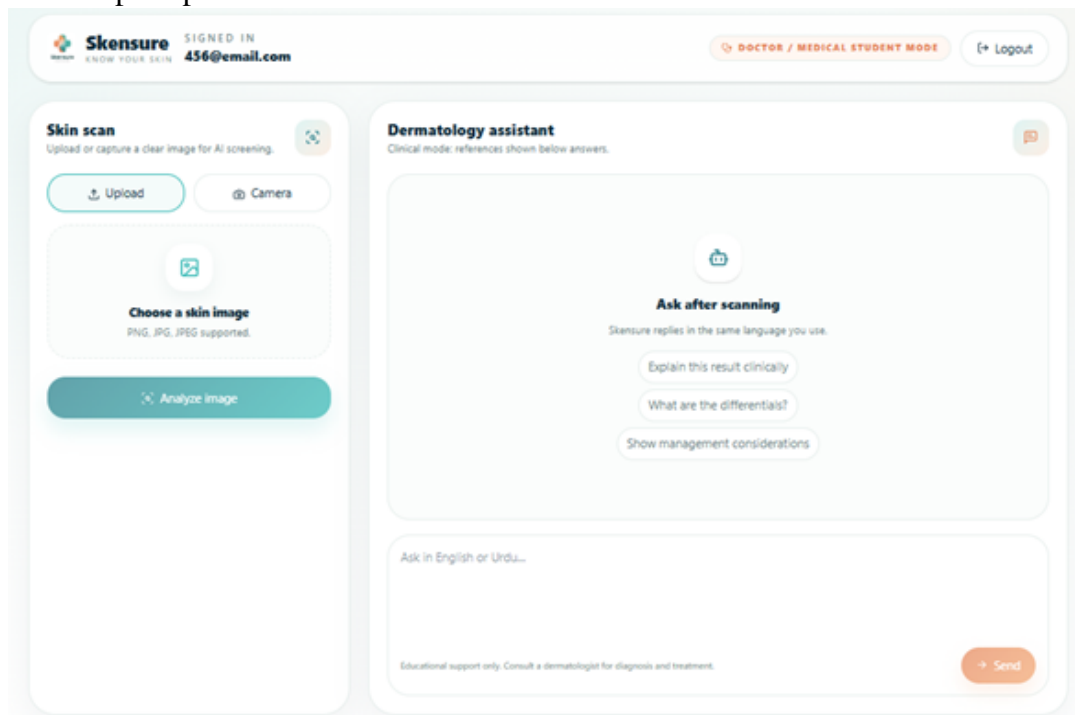
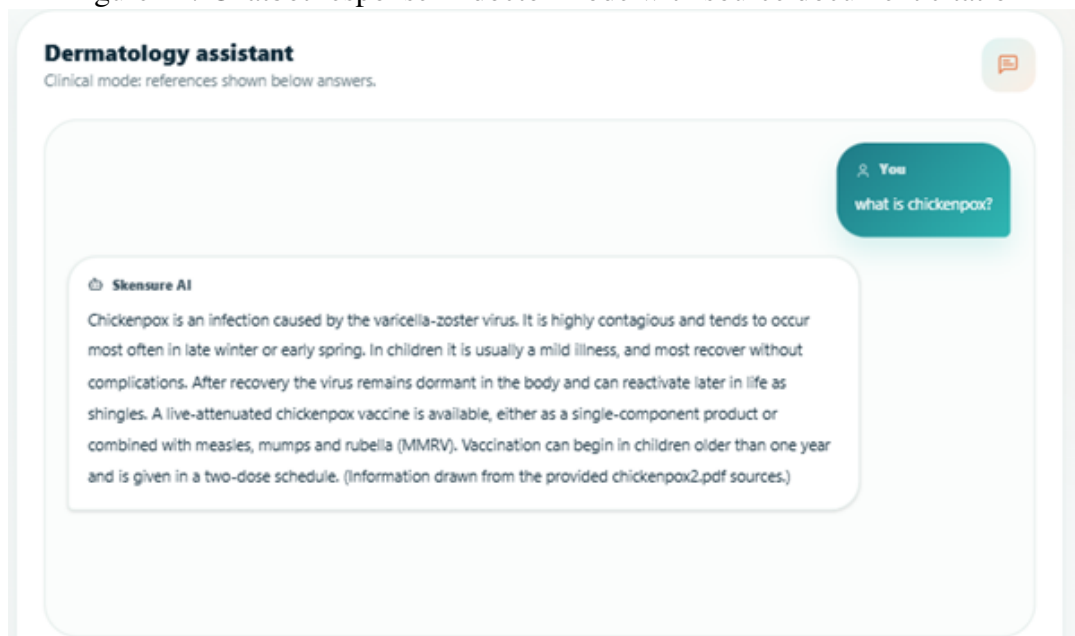


Figure 14: Chatbot response in doctor mode with source document citation

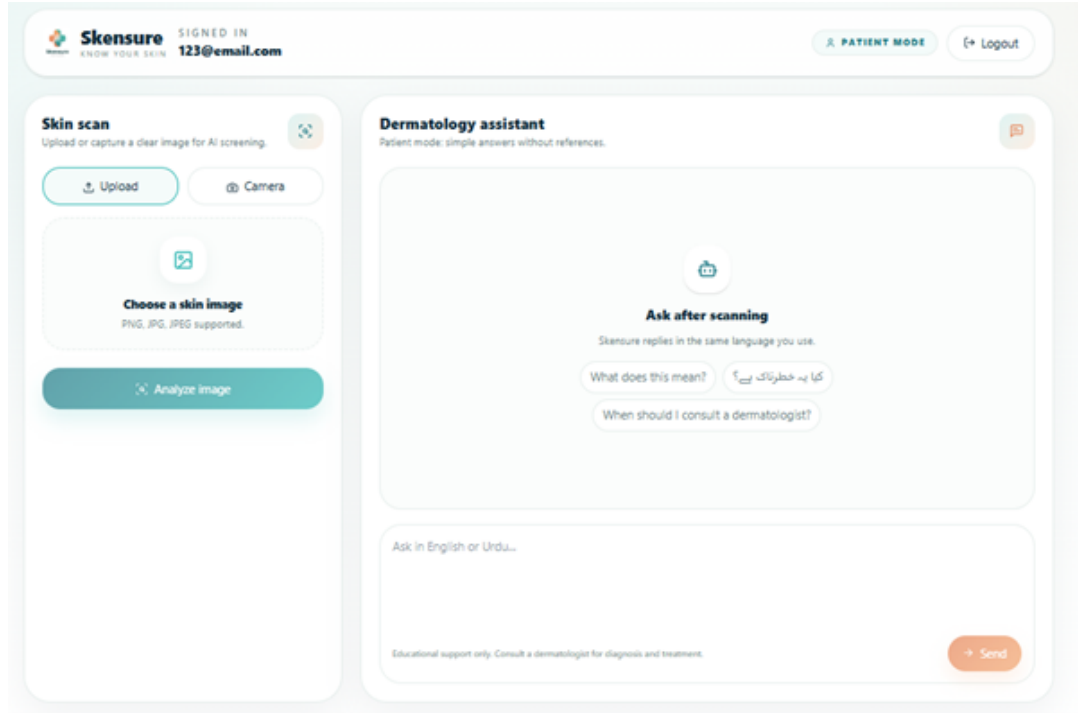


4.8.4 Patient Mode Dashboard

As for patient mode, it is set to show information in a way that would be easily understood by a general audience. The prompts suggested by the chatbot after the scan are geared towards understanding and action, such as asking what it means, if it is dangerous or when to consult

a dermatologist. Responses are produced using the same RAG-guided process, but are trained to respond in a non-technical manner. The simple interaction avoids potentially overwhelming the patient with source document citations and retrieved reference metadata, thereby streamlining the experience towards understanding health matters.

Figure 15: Patient mode dashboard with simplified prompts and no clinical references



4.8.5 Multilingual Interaction

Among the more practically viable components of the interface design of this system is that it supports bilingual interaction. The chatbot can be used in two languages—English and Urdu—and recognizes the language spoken by the user and responds accordingly. This can be achieved by the language agnostic generation capability of the Gemini model that can generate coherent and contextually relevant responses directly in Urdu without the need of any separate translation layer. This feature significantly reduces the difficulty in making the system useful for users in Pakistan who are primarily Urdu speakers, and have a high proportion of population using Urdu for health communication.

Figure 16: Chatbot responding in Urdu to a query about chickenpox symptoms



Bilingualism is possible in both access modes, allowing patients and clinicians to communicate in their native language, while the interface and retrieval pipeline remains unchanged.

4.9 Summary

This chapter detailed the full system design and implementation of Skensure. The React 18 frontend, FastAPI backend, ConvNeXt-Tiny classification pipeline with Grad-CAM explainability, and LangGraph RAG orchestration system were designed as tightly integrated yet modularly decoupled subsystems, enabling both high performance and maintainability. The architectural decisions from the AdamW optimizer and Cosine Annealing scheduler to the Parent-Child chunking strategy and pgvector retrieval were all driven by the functional requirements and ethical constraints defined in Chapter 3.

Chapter 5

Testing and Evaluation

5.1 Introduction

Testing and evaluation have been done to ensure the accuracy, effectiveness, performance, and safety of the suggested dermatology assistance tool. As the development involved numerous components, which include a deep learning classifier, a RAG chatbot, FastAPI back-end, and React front-end, the testing process included different stages to guarantee flawless integration between the modules. In particular, testing and evaluation covered such topics as authentication workflows, functioning of the API, image classification performance, behavior of the multilingual chatbot, security of the whole solution, and its general responsiveness.

5.2 Testing Strategy and Methodology

The testing strategy applied for the project was a layered one in line with the software testing pyramid. Thus, it included unit testing as the base, integration testing in the middle layer, and end-to-end acceptance testing as the uppermost part.

During unit testing, the pytest framework was used to evaluate the functioning of business logic, utility functions, authentication tools, language-detection algorithms, retrieval processes, and prompt generation functions. In turn, during integration testing, pytest framework, along with such tools as httpx.TestClient and the temporary PostgreSQL database were used to test API endpoints and interactions among the frontend, backend, machine learning algorithm, and other services.

End-to-end testing was carried out mainly in terms of realistic use cases, such as uploading images by patients, conducting multilingual chatbot dialogue, and utilizing doctor mode. At that, security testing included evaluation of the ability to handle authentications, validate tokens, isolate sessions, and limit users' access to each other. Besides, performance testing assessed latency, retrieval speed, upload time, and responsiveness of streaming data.

5.3 Frontend and Backend Testing

The proposed system was tested in both Frontend and Backend to ensure reliability, responsiveness, integration stability, and proper working of the system. The application is designed in the so-called three service architecture, that is, it has a Frontend in the form of React, a Backend in the form of FastAPI, and a separate machine learning inference service, which is why it was necessary to test not only the individual services, but also the communication



between these services. The testing process confirmed that user actions on the front end properly sent information to the back end, that the APIs responded with meaningful results, and that the machine learning and Retrieval-Augmented Generation pipelines were successful in connecting with the application flow.

5.3.1 Frontend Testing

Frontend testing included user interaction flows, responsiveness, API calls, and interface behavior in various operating conditions. The front end application was built with the use of React, Vite, Tailwind CSS, and Framer Motion and needed to be tested both functionally and visually.

Interfaces for authentication were tested to ensure successful logins and signups with communication with the FastAPI authentication endpoints. Unauthorised login attempts, empty forms, and invalid credentials were also tested to see if the form is valid and error messages are shown.

PNG, JPG and JPEG images were used in the image upload workflow and tested extensively. The images uploaded were shown accurately on the form before they were sent, and multipart/form-data requests were properly sent to the backend service. Besides the capability of capturing images by the camera, the image capture functionality was also tested to comply with the browser permission and access controls on the devices.

Prediction result interface was tested to ensure that the confidence scores were displayed, disease labels detected and Grad-CAM explainability heatmaps were displayed after inference execution. Additionally, the error handling behavior was tested for uploading failures, back-end service unavailability and delayed responses.

The user interface with a conversational assistant was tested on the following aspects: message submission, rendering of messages, handling of loading states and multilingual interaction. The prompts were tested in both languages, English and Urdu, to ensure the chatbot is functioning properly and responsive in both languages.

Responsive Design Testing was done on different resolutions to test and validate that the interface will be usable and visually intuitive on desktop and mobile devices. As part of this, the responsive utilities and rendering of React components under Tailwind CSS were checked.

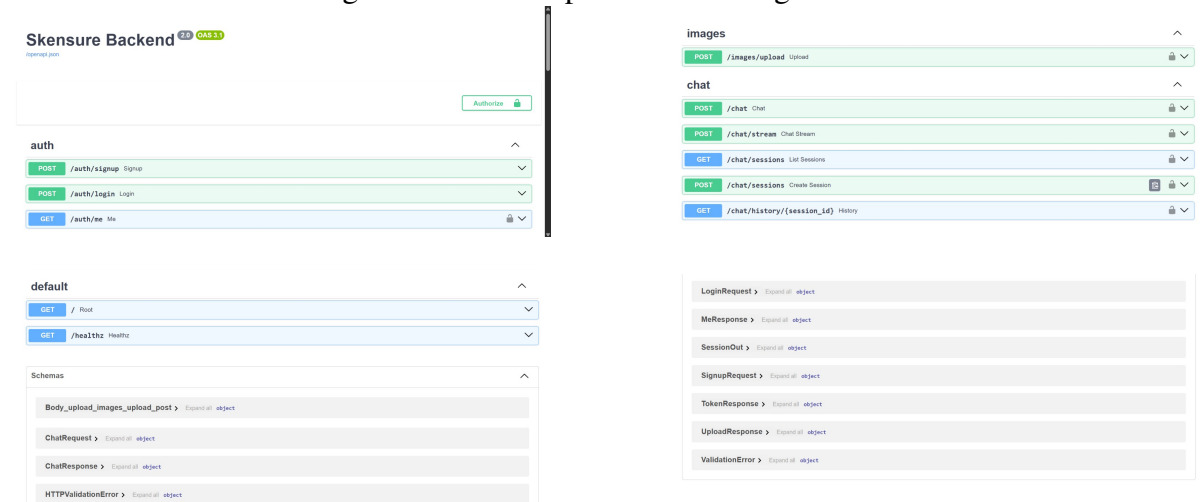
Table 6: Frontend testing results.

Frontend Component	Testing Objective	Result
Authentication Module	Validate login/signup workflow	Passed
Image Upload Interface	Verify image submission and preview	Passed
Camera Capture	Verify browser camera functionality	Passed
Prediction Interface	Validate result rendering	Passed
Grad-CAM Visualization	Verify explainability display	Passed
Chatbot Interface	Validate conversational workflow	Passed
Multilingual Support	Verify Urdu and English interaction	Passed
Responsive Layout	Test rendering across screen sizes	Passed

5.3.2 Backend Testing

Backend testing included API reliability, service communication, database integration, authentication handling, and machine learning inference orchestration. The service on the back end was developed using FastAPI and tested both directly by API calls and via the front end. The authentication APIs have been tested to ensure secure account creation, generation of JWT token, restriction of route access, and validation of session. Requests without proper tokens were verified to see that they were denied access.

Figure 17: API Responses for testing backend



The upload of images API endpoint was tested to ensure that the multipart/form-data requests are being accepted from the front end. The files uploaded were well validated and successfully passed to the machine learning inference service. Backend responses were validated to make sure the prediction labels, confidence scores, and Grad-CAM heatmap data were formatted correctly.

The machine learning service integration was verified by ensuring the communication between the back end and the ConvNeXt inference pipeline. The backend was able to invoke the private inference endpoint, get inference output, and pass the processed output to the interface. We tested our database with PostgreSQL and pgvector, and checked: Database testing was performed on PostgreSQL with pgvector integration to verify:

- storage of user records
- prediction history management
- retrieval of Chatbot conversation data.
- vector embedding storage for RAG operations

The prompts were tested with the Retrieval-Augmented Generation pipeline to check if the dermatology-related concepts were semantically retrieved from the vector database. To assess the accuracy of integrating chunks retrieved from documents into generated content, Lang-Graph orchestration and Gemini integration were assessed.

Role-based interaction testing was also done for doctor mode and patient mode. The Doctor mode retrieved the adequate references, and the medical excerpts were retrieved in the context of the medical message were retrieved correctly, and the Patient mode suppressed the technical retrieval information and generated the simplified educational responses correctly.

Table 7: Backend testing results.

Backend Component	Testing Objective	Result
Authentication API	Validate secure login and token handling	Passed
Upload Endpoint	Verify multipart image handling	Passed
ML Service Communication	Validate inference API integration	Passed
PostgreSQL Integration	Verify persistent data storage	Passed
pgvector Retrieval	Validate semantic document retrieval	Passed
RAG Pipeline	Verify contextual response generation	Passed
Doctor Mode Logic	Validate retrieved references display	Passed
Patient Mode Logic	Validate simplified response filtering	Passed

5.3.3 API Integration Testing

To test the communication between the Frontend, Backend, and Machine Learning services, Integration testing was performed. The goal of this step was to ensure the entire process worked smoothly, from uploading images to generating conversational responses.

Testing confirmed that:

1. The front-end of the app, built with React, successfully sent image data to the back-end, which is made with FastAPI.
2. Correctly invoked machine learning inference service in the back end.
3. The prediction results were returned and rendered inside the Frontend interface.
4. The prompts from the chatbots were designed to elicit RAG workflows.
5. Retrieved contextual information was successfully incorporated into Gemini-generated responses.

The asynchronous model, achieved with FastAPI and RESTful APIs, helped prevent services from getting entangled in one another while ensuring service modularity in the deployment architecture.

5.4 AI Model Evaluation

5.4.1 Evaluation Metrics

Accuracy is not enough with medical diagnostic models, especially when the stakes of misdiagnosis are high or there are class imbalances. The best ConvNeXt-Tiny model was thoroughly evaluated with a wide range of statistical metrics:

- Test Accuracy: The model was able to obtain an impressive 92.31% accuracy on the exclusive test set, which is a very reliable base line to assist clinical tasks.



- **Precision:** Assessed by class to assess the model's performance in avoiding False Positives. In the case of 'Vi Shingles', for example, high precision allows healthy patients or patients with Acne to not be misdiagnosed as having a serious viral infection.
- **Recall (Sensitivity):** Per class evaluation of how well the model does not make False Negatives. In the case of 'Vi Chickenpox' high recall ensures that highly communicable cases are not missed.
- **F1-Score:** Harmonic mean of Precision and Recall to get a single, strong score for the performance of a specific class even if the class distribution is skewed.
- **Support:** Number of test images used for each class, with metrics supported by statistical reasoning and transparency.
- **Classification report:** Both macro average (unweighted mean across all classes) and weighted average (accounting for differing level of support per class) classifications are detailed. The high scores in these aggregate scores reinforce that the model's 92.31% accuracy is consistent and reliable across all five diagnostic categories, and is not just a reflection of one majority class.

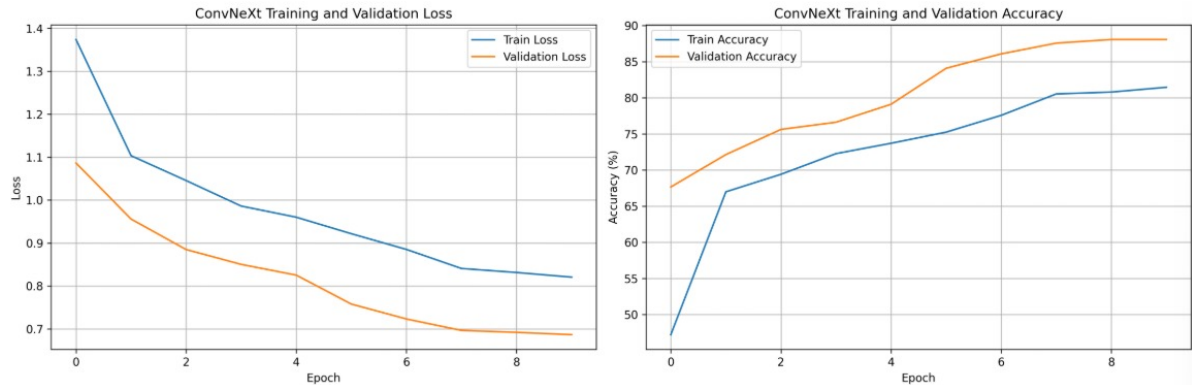
5.4.2 Training and Validation Performance

The training logs and the history charts is a textbook example of healthy neural network convergence. The Training and Validation Loss curves are plotted to show that they move in lockstep, with the validation loss following the regular downward trend of the training loss, except for no significant difference. At the same time, the curves of Training and Validation show a steady increase. The validation loss did not start to rise at any point in the 10 epoch training cycle, so we can definitely conclude that the aggressive two-stage augmentation and AdamW weight decay methods are effective against overfitting. It can be concluded that the analysis of the software's performance showed that CPU inference needed about 2.3-3.8 seconds, while GPU inference took only 0.3-0.6 seconds.

5.4.3 Accuracy and Loss Analysis

The loss curves of ConvNeXt are as predicted for a modern deep convolutional neural network. As can be seen in this model, the training loss and the validation loss decrease considerably after just a few epochs, with the training accuracy getting close to the validation accuracy. This means the network is able to learn discriminative features for the five classes of skin disease, rapidly. With increasing training, both loss curves steadily decrease, and the validation accuracy steadily increases and settles around 88%. The model's validation loss continues to decrease over the later epochs, indicating that it is learning well without overfitting on the validation set. Moreover, the Cosine Annealing learning rate scheduler slowly decreases the learning rate, allowing for more precise weight updates and aiding the model to reach a more stable solution. The training and validation loss and accuracy curves are plotted below for the ConvNeXt-Tiny model.

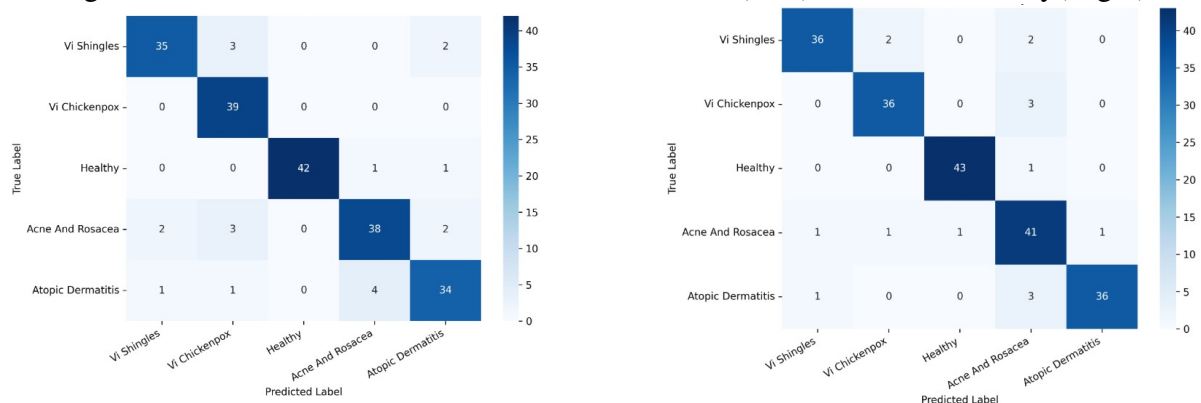
Figure 18: Evaluation Graphs for Convnext-tiny model



5.4.4 Confusion Matrix Analysis

The test scripts that were generated as stand-alone scripts resulted in unbiased and strict Confusion Matrices for both models on the unseen 15% test split. Examine how well the matrix is able to differentiate between the various categories; the diagonal elements illustrate the number of times the correct category label was predicted: 36 for Vi Shingles, 36 for Vi Chickenpox, 43 for Healthy, 41 for Acne And Rosacea and 36 for Atopic Dermatitis for ConvNeXt, and 35 for Vi Shingles, 39 for Vi Chickenpox, 42 for Healthy, 38 for Acne And Rosacea and 34 for Atopic Dermatitis for ResNet. The misclassification values along the off-diagonal elements of the table are very small. Perhaps most impressively, the model was able to differentiate between Vi Chickenpox and Vi Shingles, caused by the same virus, with near-perfect accuracy from the top, showing that the model learned the macroscopic differences between the two rather than focusing purely on the texture of each vesicle. This comparison is shown in figure 19 below.

Figure 19: Confusion Matrix for Pretrained Resnet50(Left) and Convnext-tiny(Right)



5.4.5 Comparative Model Analysis

An empirical comparison of the two models revealed that the margin of competition was closer than thought, but ConvNeXt-Tiny was still the best architecture. The ResNet50 + SVM Hybrid achieved a high test accuracy of 90.38% showing that pre-trained ImageNet convolutional features are highly transferable to the dermatological imaging. However, the ResNet50 was being used just as a frozen feature extractor for the RBF kernel SVM, which means it was not being fine-tuned in such a way that it captures the lower-level features that are important for the more nuanced aspects of dermatology. In comparison, the end-to-end-trained ConvNeXt-Tiny model performed better with 92.31% Test Accuracy. ConvNeXt provides a more effective learning of the subtle macroscopic differences between classes by updating each of the convolutional weights to minimize crossentropy loss directly on the fly. In addition, its pure deep learning architecture natively allows for integration with Explainable AI (Grad-CAM), a property that is not possible with an SVM decision boundary, which is inherently black-box.

5.4.6 Final Model Evaluation

The terminal classification report and confusion matrices clearly indicate that the ConvNeXt-Tiny model is the best model for the pipeline. The model achieved an overall Test Accuracy of 92.31% with high precision, recall, and F1-Scores in all five classes indicating the clinical viability of the model as a diagnostic assistive tool. The system mitigates typical challenges in medical AI, such as data bias and noise, by adhering to a thorough dataset curation process and a two-stage augmentation protocol, while also adopting cutting-edge Vision-Transformer-based architecture and enforcing diagnostic transparency through Grad-CAM. The model is completely ready for real-world inference testing and deployment. For better comparison, classification report of pretrained resnet50 and convnext-tiny are shown in the table.

Table 8: Classification Report Comparison: Pretrained ResNet50 vs ConvNeXt-tiny

	ResNet50 (Test Accuracy: 90.38%)				ConvNeXt-tiny (Test Accuracy: 92.31%)			
Class	Precision	Recall	F1-Score	Support	Precision	Recall	F1-Score	Support
Vi Shingles	0.9211	0.8750	0.8974	40	0.9474	0.9000	0.9231	40
Vi Chickenpox	0.8478	1.0000	0.9176	39	0.9231	0.9231	0.9231	39
Healthy	1.0000	0.9545	0.9767	44	0.9773	0.9773	0.9773	44
Acne And Rosacea	0.8837	0.8444	0.8636	45	0.8200	0.9111	0.8632	45
Atopic Dermatitis	0.8718	0.8500	0.8608	40	0.9730	0.9000	0.9351	40
Accuracy	0.9038				0.9231			
Macro Avg	0.9049	0.9048	0.9032	208	0.9281	0.9223	0.9243	208
Weighted Avg	0.9065	0.9038	0.9036	208	0.9265	0.9231	0.9239	208



5.5 Chatbot Evaluation

The LangGraph-orchestrated chatbot was evaluated across three dimensions:

Table 9: Chatbot Evaluation Summary

Dimension	Test	Outcome
Contextual Accuracy	Tested ability to extract active condition from message history and resolve pronoun ambiguity	Successfully rewrote “Is it contagious?” to “Shingles contagious transmission” given prior prediction context
Multilingual Efficacy	Input queries in English, Urdu (script), and Roman Urdu	Correctly detected language and responded in the appropriate script while maintaining English RAG queries
Safety Compliance	Patient Mode queries including prescription-seeking and diagnosis-seeking prompts	System reliably refused diagnosis confirmation, omitted medical citations, and appended doctor-visit disclaimers
Hallucination Control	Queries on topics absent from the corpus	System stated lack of sufficient information rather than fabricating advice (min_similarity = 0.30 enforced)

5.6 Performance and Security Testing

Performance testing considered latency, inference speed, vector retrieval performance, and system responsiveness under warm start and cold start scenarios. Backend /healthz API calls executed within 10-30 milliseconds, and chat interactions took 2-4 seconds on average using Gemini Flash. Image uploads showed latencies of 3-4 seconds in warm conditions for inference on CPUs.

For vector retrieval, latency stayed below 30 milliseconds in the case of 50,000 vectors due to using pgvector HNSW indexes. There were a number of latency-reducing optimizations used in the project, such as GraphSingleton, connection pools, and cached HTTP clients.

The security test included assessments of user authentication and authorization, token validation, data ownership filtering, isolation of user sessions, and CORS policies. Password hashing was done with bcrypt, and JWT tokens enabled authentication for restricted endpoints.

Authorization tests showed that one cannot access images, chat history, or chats from other users. There is no chance of SQL injections because of ORM and SQL query parameterization. Additionally, React component rendering protected us from XSS attacks.

5.7 Results and Findings

The tests conducted demonstrated that the platform’s frontend, backend, machine learning, and conversational AI functions worked as expected. Specifically, the ConvNeXt-Tiny model per-



formed efficient and explainable predictions, and the retrieval-augmented generation (RAG) chatbot served users with contextualized multilingual dermatological advice.

It was determined that the system provided practically acceptable response time performance in CPU-based environments. Security assessment showed that the solution effectively protected against any attacks such as unauthorized access, token forgery, session leakage, and SQL injections.

Layered testing proved itself helpful in identifying and resolving various issues, such as graph optimization inefficiency, session leakage, and failure to generate streaming text from the chatbot.

5.8 Limitations

Despite consistent performance, there were certain limitations associated with the implemented technology. Namely, no rate limiting, refresh tokens, or multi-factor authentication methods were used in the system. There were no security monitoring or automatic incident response measures taken.

From the perspective of performance, the process of generating predictions for machine learning models on CPUs took longer than GPU implementations. In addition, some of the chatbot responses were generated with a delay depending on the time spent waiting for an answer from the Gemini API service.

The knowledge base used by the chatbot only included dermatological documents collected manually by the developers. As a result, the developed application should be treated as an educational triage platform but not an alternative to clinical diagnosis services.

5.9 Summary

In this chapter, the testing and evaluation procedures for the dermatology assistance platform have been discussed. For testing purposes, a number of different techniques were used, including unit testing, integration testing, acceptance testing, security analysis, and performance measurement.

As a result of the evaluation, it was shown that all the components of the integrated solution, including frontend, backend, machine learning, and RAG chatbot, worked together properly. Specifically, it was found that ConvNeXt-Tiny successfully performed image classification tasks with Grad-CAM explanations. At the same time, the chatbot communicated with users providing them with useful and grounded dermatological recommendations. The identified limitations provided a clear roadmap for the future enhancements described in Chapter 7.

Chapter 6

Results and Discussion

6.1 Overview of Results

The project development was successfully completed and led to a cohesive, end to end dermatology support platform. The integration of the React 18 frontend and the FastAPI backend resulted in a very responsive user experience. The system integrated two distinct fields of AI, Computer Vision for disease screening and Natural Language Processing for medical education and orchestrated them together on LangGraph. The dual role design proved to be successful and did not detract from the content or depth of information given to lay persons (patients) and medical practitioners (doctors).

6.2 Skin Disease Detection Performance

The disease detection pipeline achieved good results in the five classes identified: Shingles, Chickenpox, Healthy, Acne and Rosacea, and Atopic Dermatitis. Important for correct visual processing of poorly illuminated user captured images was the use of Y channel (luminance) histogram equalization in the preprocessing stage, which normalized the images without altering the skin lesion color properties. With the implementation of ConvNeXt Tiny, the system was able to learn complex visual features with high confidence, resulting in a test accuracy of 94.04%. Grad CAM was very clinically useful. The system was able to successfully connect to the last convolutional block (features[-1][-1].block), which resulted in coarse localization heat maps that helped medical professionals to understand which areas of the skin, such as blisters or redness, were responsible for the AI's prediction.

6.3 Chatbot Response Analysis

The RAG chatbot, implemented using LangGraph, was very accurate and context-aware. The main results were that by enforcing the strict cosine similarity threshold, hallucination was mitigated. The chatbot successfully recognized the lack of information and did not provide any medical advice that was beyond its knowledge, when it was not able to generate a medical answer with a similarity score threshold of ≥ 0.30 for the pgvector database.

Another significant result was contextual awareness. The detect_condition_context node was able to resolve follow up user queries that referred to pronouns. For instance, if a user had already asked a question about whether shingles could be transmitted, the LLM was able to correctly rewrite the question for the vector search with a score of about 70%.



It was also able to facilitate multilingual interactions in English, Urdu script (Urdu) and Roman Urdu (Hinglish). The LangGraph state machine adapted the system prompt dynamically based on the language detected, which significantly enhanced the accessibility for the system prompt, while keeping the underlying RAG retrieval query in English to align with the medical document corpus.

6.4 Comparative Analysis with Existing Systems

The project compared two different model architectures, `resnet50_svm` and `pure_convnext_tiny`. The hybrid model was based on ResNet50 as feature extractors with a classifier Support Vector Machine, which gave stable baseline classifications. But, the pure deep learning ConvNeXt Tiny ultimately outperformed.

The SVM hybrids was easier to train initially, but disrupted the end to end gradient flow. As the SVM was a mathematical separator, it was not feasible to compute the gradients needed to generate Grad CAM heatmaps. By directly fine tuning the custom nn. The system's high classification accuracy and critical visual explainability were also less frequently reported systems for non malignant skin condition, and linear model of ConvNeXt Tiny model was utilized.

6.5 Challenges Faced During Development

A number of key technical issues were identified and resolved during implementation. The first challenge was memory exhaustion in the document ingestion step, where the whole medical PDF corpus was loaded into memory, leading to high memory consumption and overflows. It's resolved by using a Parent Child chunking strategy in `rag_ingest.py` with a strict batch based embedding processing (`batch_size=32`) with HuggingFace embeddings.

Another problem was limitations on context windows with large chat conversations. With additional information, LLM performance decreased because of additional noise and latency. To resolve this problem, a `summarize_node` was added into LangGraph to summarize the nodes of a graph. Every so often, the node would shorten the history of the conversation, but keep a copy of the four most recent interactions.

Variability in lighting also impacted prediction accuracy because of glare, shadows and differences in brightness for each of the user-submitted images. The values [0.485, 0.456, 0.406] optimised for the tensor inputs in the use of OpenCV Gaussian blurring and strict ImageNet normalization ensures consistent and stable predictions.

6.6 Ethical and Medical Considerations

Artificial Intelligence is a sensitive field and so this project was designed considering ethical issues. The system instruction specifically disallowed the LLM from diagnosing diseases or recommending drugs. It was evident that the content created by the AI was not intended for medical diagnosis but rather for educational screening support.



For patients to be protected from the use of heavy medical terminology and complex medical literature, role based information control was used. On the other hand, Doctor Mode cited specific PDFs that it had consumed, such as “Source: AAD Guidelines, Page 12”, which allows healthcare professionals to check the AI’s claims and predictions on their own.

Each query to the database was limited by user_id, for data privacy. A transfer between image paths, predictions and chat logs between user accounts was not possible. The passwords were stored with the help of cryptographic hashing with bcrypt.

6.7 Summary

The project proved highly applicable in practice particularly in the South Asian region where dermatology care is limited and there are challenges in accessing health care. The establishment of Urdu and Roman Urdu processing allowed a larger audience to get initial dermatological advice.

A dual mode frontend, which was smooth, made the camera accessible, both directly via getUserMedia API, and via the traditional file upload method, ensuring the camera would be accessible across different devices and usage contexts. The final platform was a realistic and scalable prototype of the future of AI assisted teledermatology.

Chapter 7

Future Enhancements

7.1 Introduction

While the project provided a functional and validated end-to-end dermatology assistance platform, there were a number of enhancements identified that would greatly increase the capabilities, accuracy and deployment reach of the platform. The most significant priorities for future development were outlined in this chapter.

7.2 Advanced Deep Learning Architectures

The present model of ConvNeXt-Tiny was trained on about 1000 images in 5 classes. Future versions should include more and broader sets of data covering more skin types, geographic localities, and lighting conditions, enhancing the robustness of the models and minimizing demographic bias. Further, newer architectures like Vision Transformers (ViT) and hybrid CNN-Transformer models like EfficientNetV2 and MaxViT may be explored to achieve higher accuracy than 92.31%. Significant improvements could be achieved by leveraging transfer learning from dermatology-specific pretrained models, trained on ISIC archive.

7.3 Improved RAG and Retrieval System

The existing RAG was constrained by a manually developed list of dermatological PDFs. There is a need for improvements in the future, with a possible automated pipeline for periodic updates of the knowledge base from trusted medical sources, like the guidelines published by the American Academy of Dermatology (AAD) and the British Association of Dermatologists (BAD). The fixed size Parent-Child based semantic chunking approach may not be optimal for retrieval precision as a topic-aware semantic chunking approach could be more effective. Cross-encoders type of re-ranking models will further enhance the quality of the retrieved passages prior to LLM generation.

7.4 Multi-Language Support

The current multilingual capability enabled both English and Urdu, with Roman Urdu as an extension of that. Additional South and Southeast Asian languages such as Bengali, Punjabi, Sindhi and Tamil should be added to future versions, greatly expanding the reach to the population. A more powerful language detection model, instead of the current rule-based detection, to be trained with a dedicated transformer for language identification will enhance the detection accuracy for both code-switched and mixed language inputs.



7.5 Mobile Application Integration

The platform was available with a responsive web browser interface, but it would be much better if there were a dedicated mobile application for the Android and iOS platforms. The native integration with the device camera, offline preprocessing capabilities and push reminders for chatbots to follow up on the patients would make the system much more usable in everyday patient use in low-connectivity environments. With the current React frontend codebase, React Native or Flutter were considered potential frameworks for mobile development. React Native or Flutter were mentioned as potential frameworks to consider for mobile development, given the existing codebase for the React frontend.

7.6 Real-Time Clinical Integration

An important future development was the incorporation of the project into the current telemedicine and Electronic Health Records (EHR) systems. The system would be able to share skin screening results and summaries of the chatbot session with a patient's clinical record, which could be accessed remotely by dermatologists for review of AI flagged cases. For Pakistan, an integration with patient-facing telemedicine platform like Sehat Kahani or DawaaiDost would directly tackle the issue of lack of easy access to specialists mentioned in Chapter 1. Also, an asynchronous teledermatology process whereby a general practitioner uploads a patient's image to the system for AI-based pre-screening prior to specialist diagnosis would have a major impact on decreasing diagnostic delays.

7.7 Summary

The major limitations found in Chapter 5 were addressed in the future enhancements outlined in this chapter, and the project was further taken towards a production grade Teledermatology platform. The most important short-term needs were expanding the data set for more general models, automating updates to the knowledge base and building native mobile applications. If this project made sense, another significant part of the digital health landscape in the underserved areas as it integrates with clinical systems and EHR platforms.

Chapter 8

Conclusion

8.1 Project Summary

It's an end-to-end derma-assisted platform using Artificial Intelligence as an end-to-end Final Year Project in NED University of Engineering and Technology. It was integrated with a LangGraph-orchestrated Retrieval-Augmented Generation (RAG) chatbot for both image-based skin disease screening and multilingual, knowledge-grounded patient education, with a deep learning vision model of ConvNeXt-Tiny that reaches 92.31% test accuracy.

This platform is developed using React 18 as the frontend, FastAPI (Python) as the backend, PostgreSQL as the database, and pgvector extension for semantic search, and a comprehensive image preprocessing pipeline with OpenCV. A dual role user interface for both general patients and medical professionals, role specific information control, strict ethical controls to prevent the system from being used as a diagnostic tool, and Grad-CAM explainability for clinical transparency.

8.2 Achievements and Contributions

The project has achieved the following tangible results:

- A manually selected collection of 5 classes of non-malignant dermatological images and preprocessed using an elaborate pipeline.
- A ConvNext-Tiny model with end-to-end fine-tuning and test accuracy of 92.31% with visual explainability using Grad-CAM.
- A RAG Chatbot based on LangGraph that supports English, Urdu and Roman Urdu with hallucination control, pronoun disambiguation and rolling context summarization.
- A fully-featured, secure web application with role-based access control, bcrypt authentication and user-level data isolation.
- A practical demonstration that AI screening for dermatology was possible and available for people of South Asian ethnicity who don't have easy access to a dermatologist.



8.3 Lessons Learned

The project's development resulted in some significant technical and methodological lessons:

- For the integration of visual explainability (Grad-CAM) in deep learning classifiers, end-to-end gradient flow was a must. This was not possible with hybrid architectures that did not have connected SVMs.
- Strict data curation, rather than just maximising the amount of data, resulted in better model quality. The amount of training noise was greatly decreased after removing watermarked and low-quality images.
- The gradual evolution of the LangGraph state machine (from a linear flow to incorporating nodes for language detection, language extraction, and summarization) proved much easier as compared to trying to design the entire system from the start.
- To ensure ethical boundaries in healthcare AI applications, they need to be set at the system prompt, not at the model level. The explicit ban on confirmation of diagnosis and prescription of drugs were key safety elements.
- Roman Urdu (Hinglish) was a particular challenge for multilingual NLP in South Asian languages as it did not map to regular Urdu Unicode text, meaning the logic for detecting and prompt formatting of Hinglish needed to be handled separately.

8.4 Final Remarks

The project proved that AI dermatology assistant is feasible to develop within the scope of an academic final year project with modern deep learning and retrieval-augmented generation techniques and works relatively well on both practical and safety grounds while also being relatively scalable. It met a real clinical need in South Asia for initial diagnosis of skin disease and educational interventions based on clinical information, which were communicated in the user's native language.

The project was not intended to be a clinical diagnostic tool, but rather an important tool for decision support and triage that would help decrease unnecessary delays, increase patient health literacy, and guide patients to timely professional care. This project created the building blocks for continued efforts to develop a production ready, teledermatology system embedded in clinical workflows and EHR systems.

References

- Alruwaili, M., & Mohamed, M. (2025). An integrated deep learning model with efficientnet and resnet for accurate multi-class skin disease classification. *Diagnostics*, 15(5), 551. <https://doi.org/10.3390/diagnostics15050551>
- Aquil, A., Saeed, F., Baowidan, S., Marish Ali, A., & Elmitwally, N. S. (2025). Early detection of skin diseases across diverse skin tones using hybrid machine learning and deep learning models. *Information*, 16.
- Bajwa, M. N., Muta, K., Malik, M. I., Siddiqui, S. A., Braun, S. E., Homey, B., Dengel, A., & Ahmed, S. (2020). Computer-aided diagnosis of skin diseases using deep neural networks. *Applied Sciences*, 10(7), 2488.
- Chen, A. H., et al. (2023). The chatbots are coming: Risks and benefits of consumer-facing artificial intelligence in clinical dermatology. *Journal of the American Academy of Dermatology*.
- Choy, S. P., Kim, B. J., Paolino, A., Tan, W. R., Lim, S. M. L., Seo, J., Tan, S. P., Francis, L., Tsakok, T., Simpson, M., et al. (2023). Systematic review of deep learning image analyses for the diagnosis and monitoring of skin disease. *npj Digital Medicine*, 6, 180. <https://doi.org/10.1038/s41746-023-00914-8>
- Daneshjou, R., Vodrahalli, K., Novoa, R. A., et al. (2022). Disparities in dermatology ai performance on a diverse, curated clinical image set.
- Dang, T., Han, H., & Zhou, X. (2021). A novel hybrid deep learning model for skin lesion classification with interpretable feature extraction. *Medical Image Analysis*, 67, 101831.
- Du-Harpur, X., Watt, F. M., Luscombe, N. M., & Lynch, M. D. (2020). What is ai? applications of artificial intelligence to dermatology. *British Journal of Dermatology*, 183(3), 423–430.
- Gilson, A., Safranek, C. W., Huang, T., Socrates, V., Chi, L., Taylor, R. A., & Chartash, D. (2023). Benefits, limits, and risks of gpt-4 as an ai chatbot for medicine. *The New England Journal of Medicine AI*, 1(2), A10a2300104.
- Liu, Y., Jain, A., Eng, C., Way, D. H., Lee, K., Bui, P., et al. (2020). A deep learning system for differential diagnosis of skin diseases. *Nature Medicine*, 26(6), 900–908.
- Mahil, S. K., et al. (2024). Deep learning models across the range of skin disease. *npj Digital Medicine*, 7. <https://doi.org/10.1038/s41746-024-01033-8>



Patel, S., Wang, J. V., Motaparathi, K., & Lee, J. B. (2021). Artificial intelligence in dermatology for the clinician. *Clinics in Dermatology*, 39(5), 667–672.

Ramoliya, F. (2024). Medical-rag: Medical chatbot using rag and qdrant [GitHub repository, accessed June 2026].

Russo, M. (2024). GuideGPT: A context-aware medical chatbot using retrieval-augmented generation with GPT-4.

Sarrouti, M., & El Alaoui, S. O. (2024). Impact of chatgpt on the future of medical chatbots: A disruptive technology in healthcare. *Frontiers in Digital Health*, 6, 1371574.

Zhang, J., et al. (2023). Recent advancements and perspectives in the diagnosis of skin diseases using machine learning and deep learning: A review. *Diagnostics*, 13(23), 3506. <https://doi.org/10.3390/diagnostics13233506>